

Programação Orientada a Objetos

VESPERTINO

Prof. Ricardo Pupo Larguesa

Aula 01 - 10/02/2026

Apresentação e análise de perfil

Ricardo Pupo Larguesa

Professor e Empreendedor na área da Tecnologia
Santos, São Paulo, Brazil



Summary

Professor no curso de Análise e Desenvolvimento de Sistemas da FATEC; Empreendedor co-fundador da T2S e das StartUps EvoluRP, DataRecintos e Ponctual; Escritor autor de Engenharia de Prompt para Devs e colunista no Jornal A Tribuna; Criador de conteúdo nos canais Tecnologia Portuária e Reprogramando.

Experience

T2S
Founder
February 2003 - Present (21 years 7 months)
Santos, São Paulo, Brazil

Sócio-fundador e Diretor

Tecnologia Portuária
Founder
August 2022 - Present (2 years 1 month)
Santos, São Paulo, Brazil

Fundador do Canal e co-host do PortCast

Faculdade de Tecnologia de Praia Grande (Fatec)
University Professor
August 2006 - Present (18 years 1 month)
Praia Grande, São Paulo, Brazil

Jornal A Tribuna de Santos
Freelance Columnist
November 2021 - Present (2 years 10 months)
Santos

Education

FGV - Fundação Getulio Vargas
MBA, International Administration · (2008 - 2009)

Universidade Federal de Uberlândia - UFU
postgraduate, Computer Engineering · (2002 - 2004)

Universidade Santa Cecília
Engineer, Information & Technology · (1997 - 2001)



Ricardo Pupo Larguesa
IA & Machine Learning para Negócios |
Empreendedor e Professor Tech | Autor
do livro "Engenharia de Prompt para Devs"



Grupo de comunicação



FPG-ADS-POO-2026-1-V

Grupo do WhatsApp



O QR code deste grupo é privado. As pessoas com quem o QR code for compartilhado poderão usar a câmera do WhatsApp para escaneá-lo e entrar no grupo.



**E vocês, quem
são?**

Projeto pedagógico

- **Disciplina:** Programação Orientada a Objetos
 - **Ementa:** Conceitos e evolução da tecnologia de orientação a objetos. Limitações e diferenças entre o paradigma da programação estruturada em relação à orientação a objetos. Conceito de objeto, classe, métodos, atributos, herança, polimorfismo, agregação, associação, dependência, encapsulamento, mensagem e suas respectivas notações na linguagem padrão de representação da orientação a objetos. Implementação de algoritmos orientado a objetos utilizando linguagens de programação. Aplicação e uso das estruturas fundamentais da orientação a objetos.
 - **Objetivo:** Implementar de softwares com o uso de uma linguagem de programação orientada a objetos.

Plano de ensino

- 1 Apresentação e Avaliação de perfil -> Apresentação da disciplina e do plano de ensino; Avaliação de perfil da turma.
- 2 Java: Introdução -> Java SE e kits de desenvolvimento: OpenJDK, Oracle JDK e GraalVM Editores de código e IDEs: IntelliJ, VSCode, Eclipse e Netbeans Gerenciadores de dependências: Maven, Gradle e Ant Agentes de Codificação: GEMINI CLIP Preparação de um ambiente para aplicações de console
- 3 Java: Revisão -> Revisão: sintaxe básica do Java, tipos de dados, laços de repetição.
- 4 Java: Introdução a OO -> Classes, Atributos e Métodos
- 5 Java OO -> Objetos, instâncias e construtores
- 6 Java OO -> Herança, polimorfismo, classes abstratas e interfaces
- 7 Avaliação A1 -> Avaliação do primeiro bimestre
- 8 Revisão da A1 -> Revisão da P1 e reforço utilizando assistente de IA especialista.
- 9 Java OO -> Tratamento de exceções
- 10 Java OO -> Uso de Collections (List, Set, Map, Queue/Deque)
- 11 Arquitetura web -> HTTP, REST, arquitetura cliente-servidor
- 12 JavaEE: Introdução -> Introdução ao Java Web e Javalin
- 13 Projeto Semestral -> Definição de tema para projeto semestral e elaboração de cronograma.
- 14 Java: JDBC -> Persistência, JDBC e Controle de Acesso
- 15 Projeto -> Projeto: To-do list
- 16 Projeto -> Projeto: WebParkControl
- 17 Projeto -> Projeto: MyFinance\$
- 18 Projeto -> Desenvolvimento
- 19 A2 -> Avaliação do segundo bimestre (apresentação dos projetos)
- 20 Prova substitutiva -> Prova substitutiva

Metodologia

- Exposição de conceitos e exemplos
- Prática laboratorial utilizando a plataforma Java
- Execução de projetos em Java aplicando conceitos de Orientação a Objetos
- Utilização de codificação agêntica usando o Gemini CLI
- Testes bimestrais para avaliação de desempenho

Plano de recuperação e exame

- Revisão e reforço das avaliações bimestrais assistida por IA
- Monitoria
- Entrevista bimestral e chamada oral para reavaliação
 - Orientação, apoio e aconselhamento individual

Reprogramando

@reprogramando 1,29 mil inscritos 125 vídeos

O mundo PRECISA de programadores! 🍌 >

Inscrito

INÍCIO

VÍDEOS

SHORTS

PLAYLISTS

COMUNIDADE

CANAIS

SOBRE

Vídeos mais acessados

Reproduzir tudo

LÓGICA DE PROGRAMAÇÃO

INTRODUÇÃO

13:53

Introdução à Lógica de Programação na linguagem...

1,7 mil visualizações • há 5 meses

Java EE | Servlet

AMBIENTE DESENV.

19:31

Preparação de ambiente de desenvolvimento JavaEE e...

1,3 mil visualizações • há 5 meses

PROGRAMAR É DIFÍCIL?

7:17

Aprender a programar é difícil? [LÓGICA DE...

877 visualizações • há 5 meses

StackBlitz

IDE ONLINE GRATUITA

14:35

Primeiros Passos com o StackBlitz: Introdução e...

775 visualizações • há 5 meses

ENGANANDO O ChatGPT

9:59

Burlando o ChatGPT [Prompt Injection]

673 visualizações • há 4 meses

VARIÁVEIS E OPERADORES

Lógica de Programação

14:27

Variáveis e operadores [LÓGICA DE PROGRAMAÇÃO]

563 visualizações • há 4 meses



Java EE | Servlet

AMBIENTE DESENV.



Curso de introdução ao JavaEE (2023)

Reprogramando

12 vídeos 1.528 visualizações Última atualização e...



Reproduzir t... Ordem aleat...

Este curso está em andamento. Inscreva-se no canal e ative o sininho para ser notificado sobre novos vídeos:

<https://www.youtube.com/@reprogramando>

Neste curso de introdução ao JavaEE (Java Enterprise Edition) você irá aprender a desenvolver aplicações web dinâmicas utilizando uma das tecnologia smais usadas no mercado.

Codificará servlets que processar e responder dados dinamicamente, implementará interfaces web utilizando ava Server Pages integradas a um banco de dados.

Requisitos:

- Fundamentos de Java



Preparação de ambiente de desenvolvimento JavaEE em 2023 [NETBEANS + GLASSFISH]

Reprogramando • 1,3 mil visualizações • há 5 meses



Introdução ao Java EE em 2023: Criando seu Primeiro Servlet

Reprogramando • 508 visualizações • há 5 meses



Criando Servlets Práticos no Java EE em 2023: Somando Números e Gerando Listas Aleatórias

Reprogramando • 292 visualizações • há 5 meses



Construindo um Relógio Digital com Java EE: Desenvolvendo um Servlet de Imagem Dinâmica

Reprogramando • 186 visualizações • há 5 meses



Criação de Aplicação Web em Java & Vue.js: Manipulando JSON e Implementando Lista de Strings

Reprogramando • 275 visualizações • há 5 meses



Introdução ao Java Server Pages (JSP): Desenvolvimento Web com NetBeans e GlassFish - 2023

Reprogramando • 323 visualizações • há 4 meses



Reusando código em DIRETIVAS [JAVA SERVER PAGES]

Reprogramando • 124 visualizações • há 4 meses



Controlando o acesso dos usuários no JSP [Java EE]

Reprogramando • 154 visualizações • há 4 meses



Definição do ambiente de DEV

- Java: Por que?
 - <https://www.tiobe.com/tiobe-index/>
 - <http://pypl.github.io/PYPL.html>
 - <https://www.apinfo.com/apinfo/>
- GitHub: Por que?
 - <https://happygitwithr.com/big-picture.html>
 - <https://dzone.com/articles/5-reasons-why-devs-love-github-and-microsoft-buys>



Nivelamento técnico

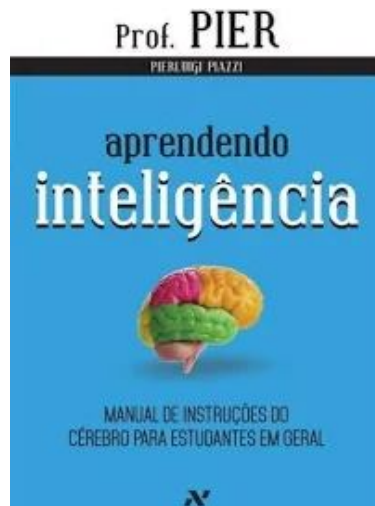


Cursos de HTML+CSS

- Mozilla:
 - <https://developer.mozilla.org/pt-BR/docs/Aprender/HTML>
- W3schools
 - <https://www.w3schools.com/html/>

Aprendendo inteligência

- Pesquise por “**pierluigi piazzi aprendendo inteligencia**” no YouTube e assista uma das palavras do professor Pierluigi
- Compre e leia o livro “Aprendendo Inteligência”



Análise de perfil da turma

Formulário:

<https://forms.gle/xBMabL5tvS2Zk4pLA>





Aula 02 - 24/02/2026

Preparação do ambiente de Desenvolvimento

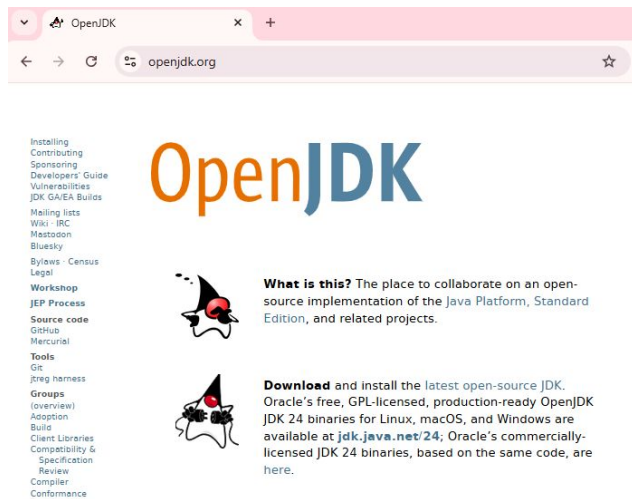
Sumário

1. Java SE e kits de desenvolvimento
 - a. OpenJDK
 - b. Oracle JDK
 - c. GraalVM
2. Editores de código e IDEs
 - a. IntelliJ, VSCode, Eclipse e Netbeans
3. Gerenciadores de dependências
 - a. Maven, Gradle e Ant
4. Agentes de Codificação
 - a. GEMINI CLI
5. Preparação de um ambiente para aplicações de console

Java SE (Standard Edition)

- **OpenJDK (Open Java Development Kit):**
implementação de referência, livre e de código aberto, da plataforma Java, Standard Edition (Java SE). Desde o Java 7, o OpenJDK é a base sobre a qual o JDK da Oracle é construído. Na prática, para a maioria dos casos de uso, o OpenJDK e o Oracle JDK são funcionalmente idênticos, especialmente desde o Java 11, quando a Oracle alinhou ainda mais os dois projetos. Para o ambiente acadêmico, o OpenJDK é a escolha padrão e mais recomendada por ser **totalmente gratuito e livre de questões de licenciamento comercial**.

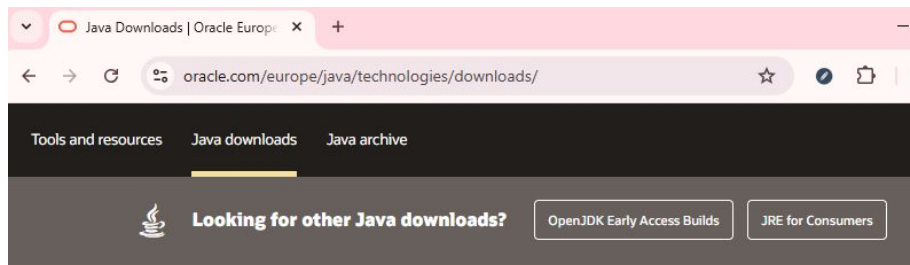
<https://openjdk.org/>



Java SE (Standard Edition)

- **JDK (comercial, como o da Oracle):**
A Oracle oferece seu próprio JDK, que até recentemente incluía alguns recursos comerciais e suporte de longo prazo (LTS) pago. Embora a Oracle tenha introduzido um modelo de licença mais permissivo (No-Fee Terms and Conditions - NFTC), para simplificar e evitar qualquer confusão de licenciamento no ambiente de ensino, o OpenJDK continua sendo a opção mais direta.

<https://www.oracle.com/europe/java/technologies/downloads/>



Java 24, Java 21, and earlier versions available now

JDK 24 is the latest release of the Java SE Platform.

[Learn about Java SE Subscription](#)

JDK 21 is the latest *Long-Term Support (LTS)* release of the Java SE Platform.

Earlier JDK versions are available below.

[JDK 24](#) [JDK 21](#) [GraalVM for JDK 24](#) [GraalVM for JDK 21](#)

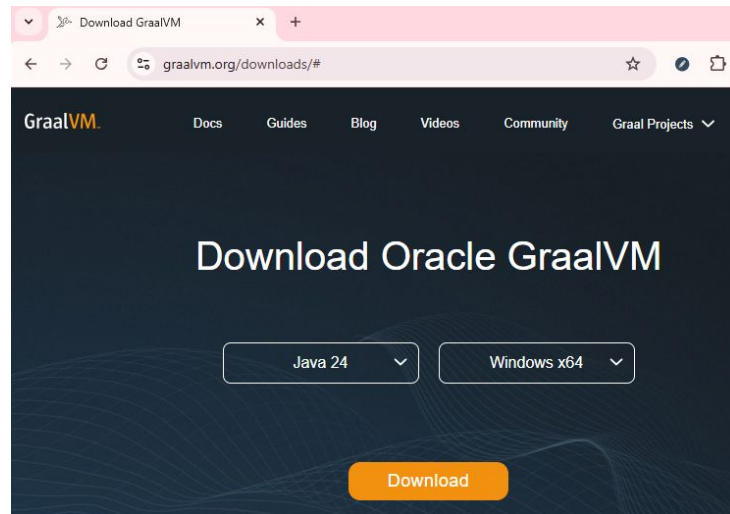
Java SE Development Kit 24.0.2 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and](#)

Java SE (standard Edition)

- **GraalVM (Graal Virtual Machine):** Lançado mais recentemente pela Oracle Labs, o GraalVM é uma extensão do JDK. Trata-se de uma JVM de alto desempenho e um JDK que não apenas otimiza a execução de código Java, mas também suporta a interoperabilidade com outras linguagens como JavaScript, Python e R. A grande inovação do GraalVM para o mundo Java é a Native Image. Essa tecnologia permite a compilação ahead-of-time (AOT) de uma aplicação Java em um executável nativo.

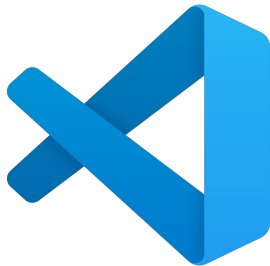
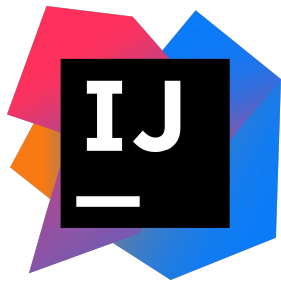
<https://www.graalvm.org/downloads>



Java SE (Standard Edition)

- **Outras distribuições**

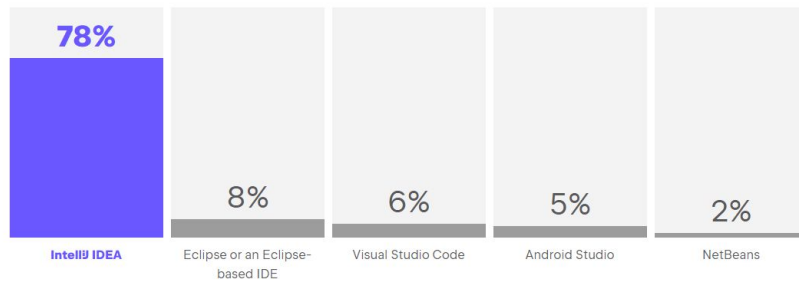
- Há várias outras distribuições do JDK baseadas no OpenJDK, como Amazon Corretto, AdoptOpenJDK, entre outras.
- Há até um build do OpenJDK da Microsoft para o Windows:
 - <https://learn.microsoft.com/pt-br/java/openjdk/download>
- Elas oferecem alternativas com diferentes características e licenças, geralmente adaptadas para plataformas específicas.



Editores de código e IDEs

jetbrains.com/lp/devecosystem-2023/java/

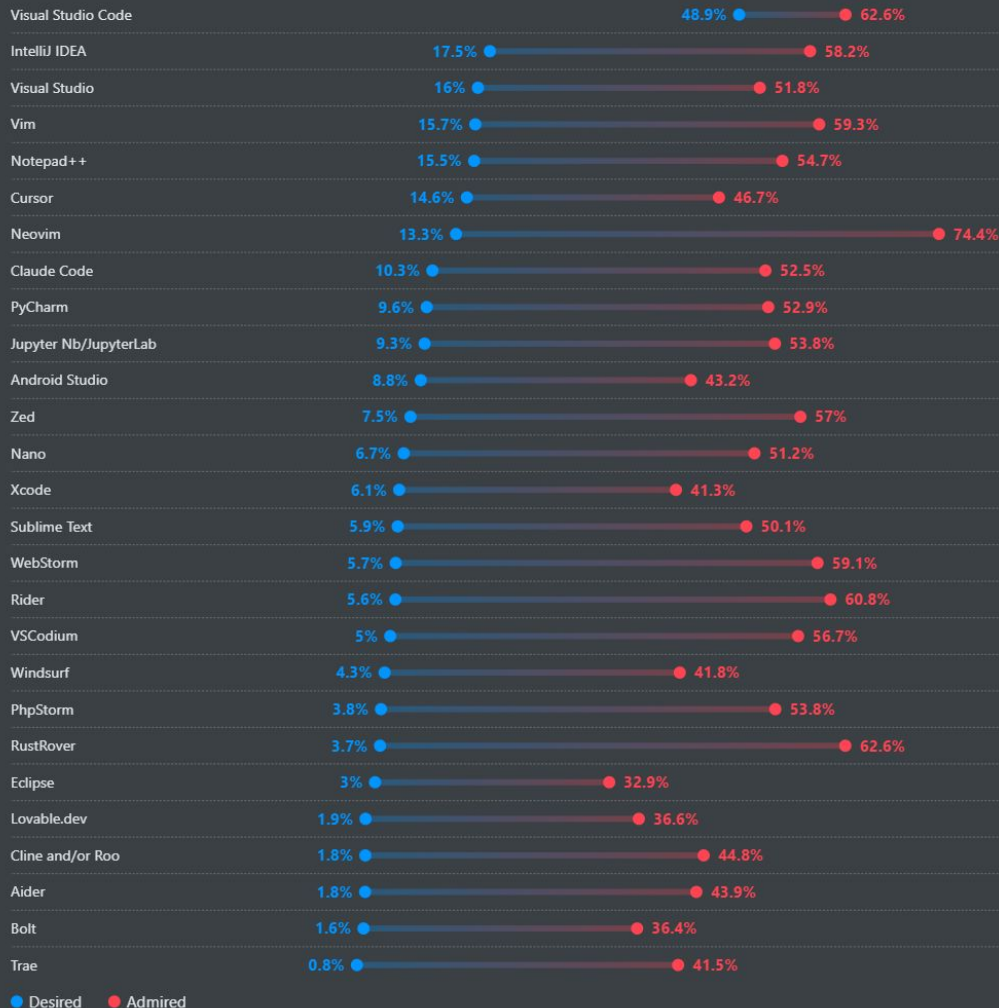
Which IDE / editor do you use the most for Java development?



Dev IDEs

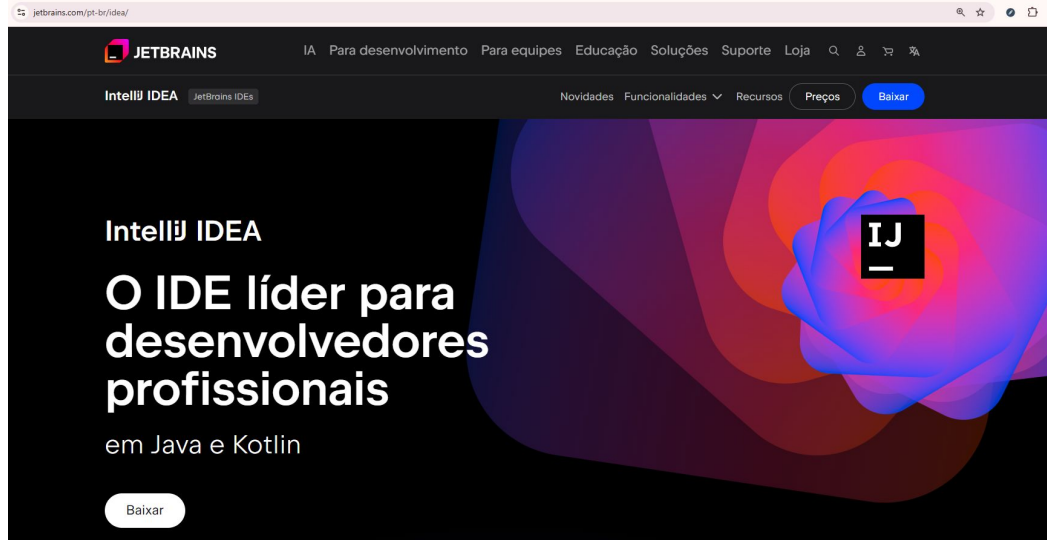
Not only is Visual Studio Code the most used IDE for five years in a row, it is also consistently ranked the most desired IDE. Developers that haven't used VS Code overwhelmingly want to use it in the next year.

? Which **development environments** and **AI-enabled code editing tools** did you use regularly over the past year, and which do you want to work with over the next year? Please check all that apply.



IntelliJ IDEA

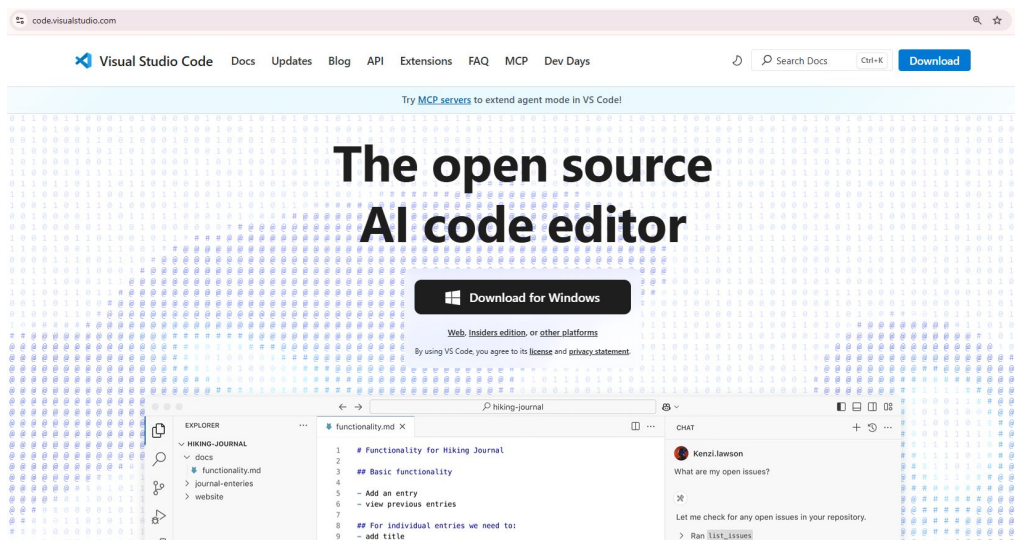
- Lançado pela **JetBrains** em **2001**, foi a primeira IDE Java a oferecer recursos avançados de refatoração de código e análise estática, que hoje são padrão de mercado.
- Por muitos anos, foi visto como uma alternativa paga e premium ao Eclipse. Com a introdução da versão Community (gratuita e de código aberto), sua popularidade explodiu. Atualmente, é amplamente considerado o líder de mercado, famoso por sua inteligência de código, ergonomia, ferramentas de build integradas (Maven/Gradle) e um ecossistema de plugins robusto. A versão Ultimate (paga) adiciona suporte avançado para frameworks de backend (Spring, Jakarta EE) e frontend.
- É a escolha predominante no desenvolvimento profissional. Sua capacidade de entender o contexto do código ("code intelligence") é inigualável, o que acelera drasticamente o desenvolvimento e a manutenção.



<https://www.jetbrains.com/pt-br/idea>

Visual Studio Code

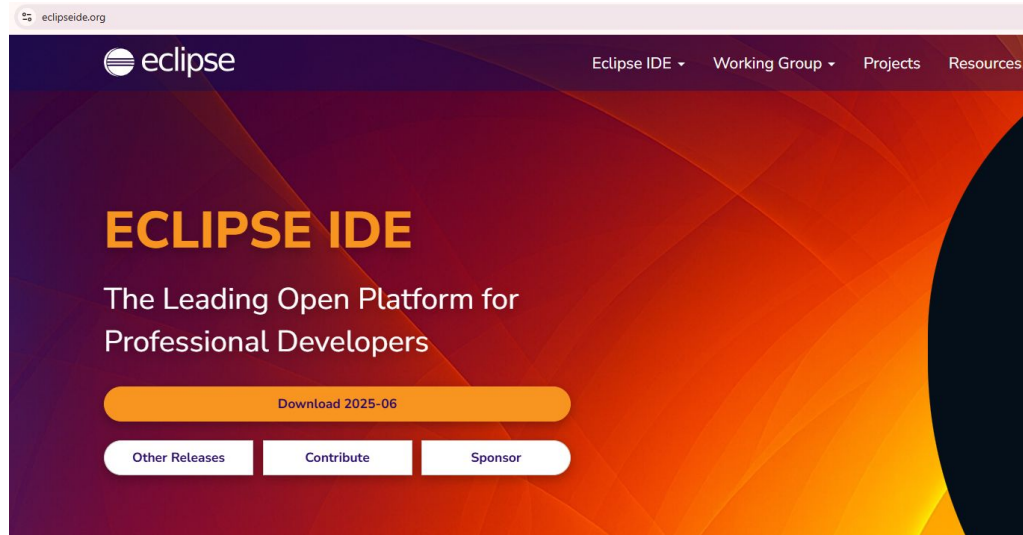
- Lançado pela **Microsoft** em **2015**, o VS Code não é uma IDE tradicional, mas um editor de código leve e extensível. Inicialmente focado em desenvolvimento web (JavaScript/TypeScript), ele rapidamente ganhou popularidade em diversas linguagens graças ao seu modelo de extensões. Com o "Extension Pack for Java" da Microsoft e da Red Hat, ele se tornou uma alternativa extremamente viável e popular para o desenvolvimento Java, especialmente para projetos baseados em Maven ou Gradle e para iniciantes.
- É a **escolha favorita de estudantes, desenvolvedores que trabalham com múltiplas linguagens** (polyglot programmers) e para quem prefere um **ambiente mais leve e rápido**. É excelente para o desenvolvimento de microsserviços e projetos com Spring Boot, Quarkus e Micronaut. Sua popularidade continua a crescer exponencialmente.



<https://code.visualstudio.com>

Eclipse IDE

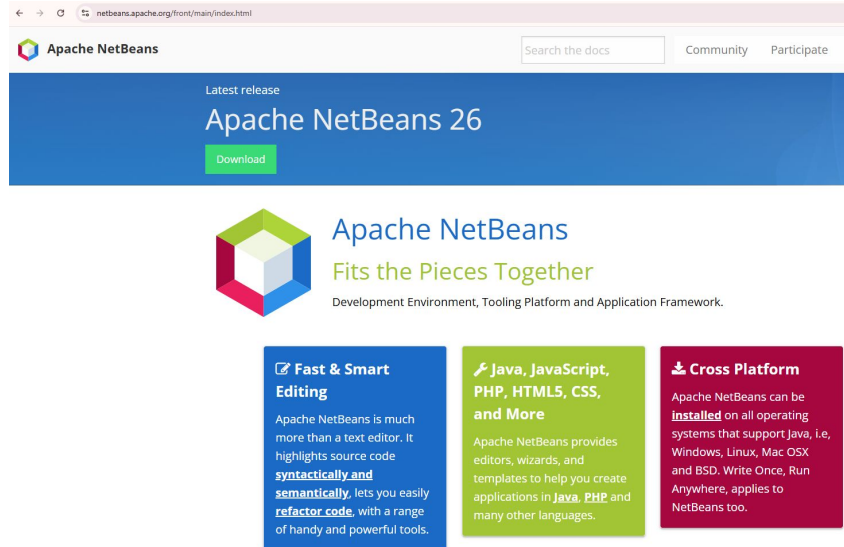
- Lançado pela **IBM** em **2001** e posteriormente gerenciado pela Eclipse Foundation, o Eclipse foi por muito tempo a IDE Java de código aberto dominante. Sua arquitetura baseada em plugins permitiu a criação de um ecossistema gigantesco para diversas finalidades. Foi fundamental na popularização do desenvolvimento Java profissional. Embora tenha perdido a liderança para o IntelliJ IDEA em termos de preferência, ainda é uma ferramenta poderosa, gratuita e muito utilizada, especialmente em projetos legados, no mundo acadêmico e em algumas grandes corporações.
- **Continua relevante** em empresas que padronizaram seu fluxo de trabalho nele há anos. Também é a base para outras ferramentas de nicho, como o Spring Tools Suite (STS) versão 3 e algumas IDEs para desenvolvimento Android (antes da migração para o IntelliJ/Android Studio).



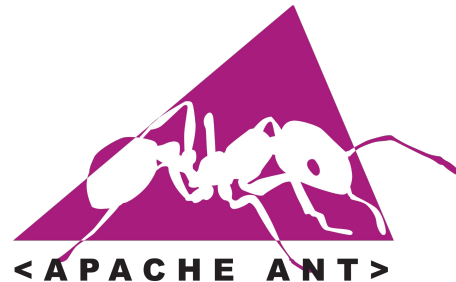
<https://eclipseide.org>

NetBeans

- Uma das IDEs Java mais antigas, com origem em um **projeto estudantil de 1996**, foi adquirida pela Sun Microsystems e se tornou a IDE "oficial" do Java por muitos anos. Era conhecida por suas excelentes ferramentas de desenvolvimento de interfaces gráficas (Swing). Em 2016, a Oracle doou o projeto para a Apache Software Foundation, o que revitalizou seu desenvolvimento. O Apache NetBeans continua sendo uma IDE de código aberto sólida, com bom suporte a Maven e Gradle e funcionalidades modernas.
- Possui uma base de usuários leal, mas sua participação de mercado é consideravelmente menor que a dos três líderes. É **frequentemente utilizado no ambiente acadêmico** devido à sua simplicidade e história ligada à própria Sun/Oracle.



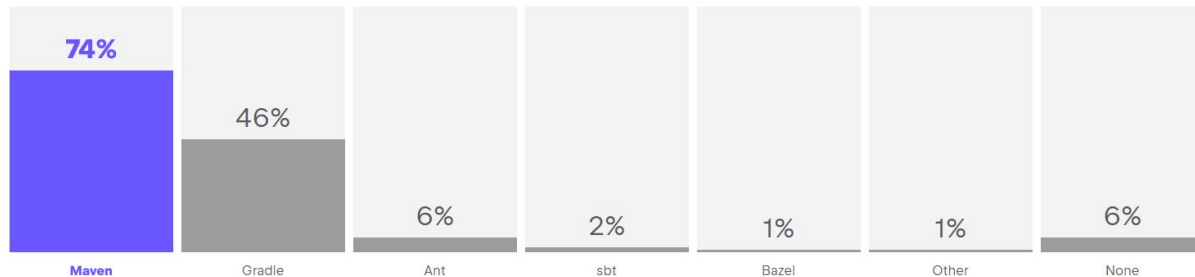
<https://netbeans.apache.org>



Gerenciadores de dependências

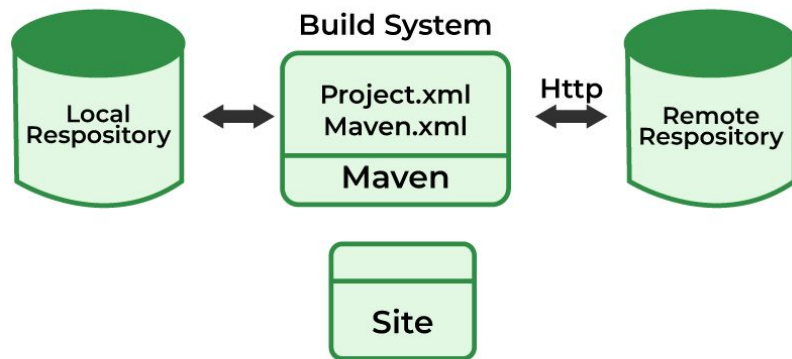
jetbrains.com/lp/devecosystem-2023/java/

Which build systems do you regularly use?



O que são e para que servem?

Os gerenciadores de dependências (ou ferramentas de build) são o pilar do desenvolvimento Java moderno. Eles automatizam o processo de baixar bibliotecas, compilar o código, executar testes e empacotar a aplicação, eliminando o caos que era o gerenciamento manual de arquivos `.jar`.



Apache Maven

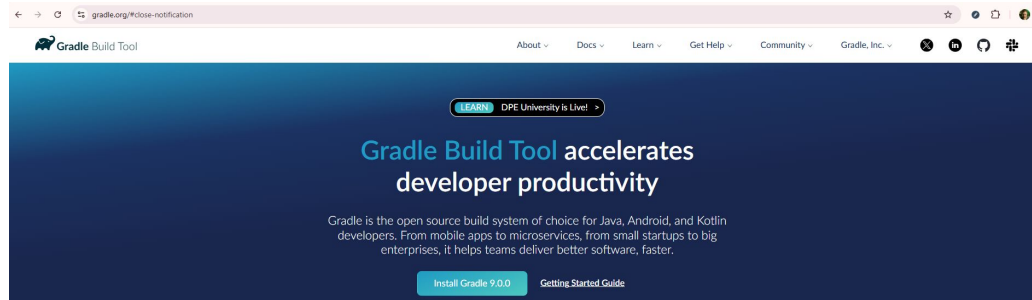


- Lançado em **2004** pela **Apache**, o Maven revolucionou o ecossistema Java ao introduzir duas ideias poderosas: o conceito de "convenção sobre configuração" e um repositório centralizado de dependências (o **Maven Central**). Antes dele, projetos usavam ferramentas como o Apache Ant, que exigia a escrita de scripts complexos e o gerenciamento manual de JARs. O Maven padronizou a estrutura de diretórios (*src/main/java*, *src/test/java*) e o ciclo de vida do build (*compile*, *test*, *package*, *etc.*). Seu arquivo de configuração declarativo, o **pom.xml** (Project Object Model), tornou a gestão de dependências e a construção de projetos drasticamente mais simples e replicáveis. Atualmente, o Maven é uma ferramenta extremamente madura, estável e ainda amplamente utilizada em todo o mercado.
- Por ter sido o padrão por tantos anos, o Maven é encontrado em uma vasta quantidade de projetos, desde sistemas legados até novos desenvolvimentos. É conhecido por sua robustez e pela enorme quantidade de plugins e integrações disponíveis. **Muitas empresas o utilizam como padrão para seus projetos Java.**
- Aproximadamente 62% de uso

<https://maven.apache.org>

Gradle

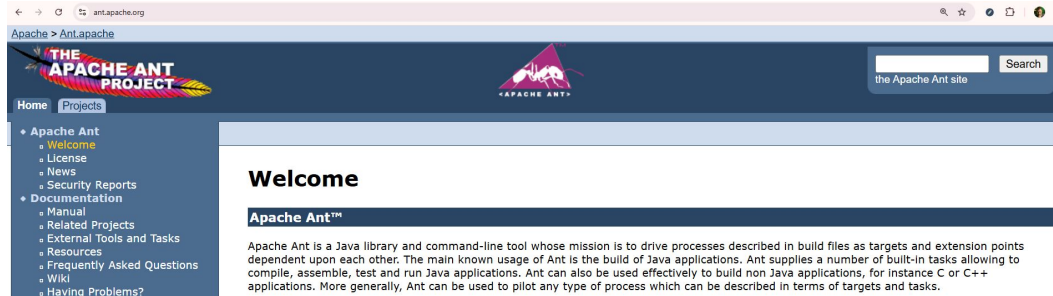
- Lançado em **2007** e ganhando tração significativa a partir de 2012, o Gradle foi criado para superar algumas das limitações do Maven, principalmente a rigidez do XML e a performance em projetos muito grandes. Em vez de XML, o Gradle utiliza uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin para seus scripts de build (*build.gradle*). Isso oferece uma flexibilidade e um poder de customização muito maiores. Suas principais vantagens são a performance superior (devido a caches avançados e daemons de compilação) e a expressividade de seus scripts de build. O Google escolheu o Gradle como a ferramenta de build padrão para o desenvolvimento Android, o que impulsionou massivamente sua adoção.
- Gradle é a **escolha predominante no desenvolvimento Android** e é muito popular em grandes projetos monolíticos ou com builds complexos, onde sua performance e flexibilidade fazem uma grande diferença. Também é amplamente utilizado em projetos de código aberto e por equipes que buscam o máximo de desempenho e customização em seu processo de build.



<https://gradle.org>

Apache Ant

- Lançado em **2000**, o Ant foi o predecessor do Maven. Ele é uma ferramenta de build baseada em scripts XML, mas não possui um sistema de gerenciamento de dependências nativo. Para gerenciar dependências com o Ant, era comum usar uma ferramenta complementar chamada Apache Ivy. O Ant oferece controle total sobre o processo de build, mas exige que o desenvolvedor defina explicitamente todas as tarefas (compilar, copiar arquivos, etc.), tornando-o muito mais verboso e menos padronizado que o Maven.
- Hoje em dia, o Ant **é considerado uma ferramenta legada**. É muito raro ver novos projetos começando com Ant. Sua presença se limita a sistemas mais antigos que ainda não foram migrados para Maven ou Gradle. Conhecê-lo é útil para arqueologia de software, mas não é uma habilidade demandada para novos desenvolvedores.





Agentes de Codificação - O que são

- Um **"agente codificador"** (ou **AI Coding Assistant**) é uma ferramenta de inteligência artificial projetada para auxiliar desenvolvedores diretamente no ambiente de programação. Ele atua como um "par programador" virtual, com a capacidade de:
 - **Autocompletar código:** De trechos simples a blocos complexos de lógica.
 - **Gerar código a partir de linguagem natural:** Você descreve o que quer em um comentário (ex: "// cria uma função para ler um arquivo CSV e retornar uma lista de produtos") e o agente escreve o código.
 - **Explicar e documentar código:** Selecionar um trecho de código e pedir uma explicação ou a geração de documentação.
 - **Encontrar bugs e sugerir correções:** Analisar o código em busca de erros lógicos ou de sintaxe.
 - **Atuar como um "chat" de conhecimento:** Fazer perguntas sobre algoritmos, bibliotecas ou boas práticas diretamente no terminal ou IDE.
- Essencialmente, ele integra o poder de um Modelo de Linguagem Grande (LLM) como o Gemini ao fluxo de trabalho do desenvolvedor.



Agentes de Codificação - História

- A ideia de assistência de código não é nova, mas a tecnologia evoluiu drasticamente.
 - **Primórdios (Anos 90-2000):** O início foi marcado por ferramentas de autocompletar baseadas em análise estática (ex: IntelliSense da Microsoft). Elas eram ótimas para sugerir nomes de métodos e variáveis, mas não entendiam a intenção do programador.
 - **Primeira Onda de IA (Anos 2010):** Ferramentas como o Kite e o Tabnine (em suas versões iniciais) começaram a usar modelos de Machine Learning mais simples para prever sequências de código com base em padrões aprendidos de milhões de projetos de código aberto. As sugestões se tornaram mais contextuais e inteligentes.
 - **A Revolução dos LLMs (2021-Presente):** O lançamento do GitHub Copilot, baseado no modelo GPT da OpenAI, mudou o jogo. Pela primeira vez, uma ferramenta podia entender linguagem natural e gerar blocos inteiros de código funcional. Isso marcou o início da era dos agentes codificadores modernos, que foram rapidamente seguidos por concorrentes como Cursor, WindSurf, Claude, ChatGPT e, claro, o Gemini do Google.



CURSOR



GEMINI CLI

🎯 **Free tier:** 60 requisições/min e 1,000 requisições/dia com conta pessoal do Google

🧠 **Powerful Gemini 3.1 Pro:** 1M de tokens na janela de contexto

🔧 **Built-in tools:** Google Search grounding, manipulação de arquivos, shell commands, web fetching

🔌 **Extensible:** Suporte ao MCP (Model Context Protocol) para integrações personalizadas

💻 **Terminal-first:** Projetado para linha de comando, funciona com qualquer editor ou IDE

🛡️ **Open source:** Apache 2.0 licensed

<https://github.com/google-gemini/gemini-cli>

github.com/google-gemini/gemini-cli

README

Contributing

Apache-2.0 license

Security



Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

> write a short paragraph about why Gemini CLI is awesome

• I will start by searching the web for "Gemini CLI" to understand its main features and purpose. Following that, I'll examine the README.md file for a project-specific overview. I will then consult docs/index.md and docs/cli/commands.md to gain a deeper understanding of its capabilities. Finally, I will write a short paragraph summarizing why Gemini CLI is awesome.

-- GoogleSearch Searching the web for: "Gemini CLI features and purpose"

! Uncovering Gemini's Awesome (esc to cancel, 21s)

Using 3 GEMINI.md files

~/code/gemini-cli (release*)

no sandbox (see /docs)

gemini-2.5-pro (99% context left)

Preparação do ambiente no VSCode

Preparação do ambiente no VSCode

Passo 0: Downloads Prévios

Antes de começar, baixe os seguintes programas. Apenas baixe, não precisa instalar ainda.

1. **Visual Studio Code:** O editor de código:
 - a. <https://code.visualstudio.com>
2. **Node.js:** O Gemini CLI precisa do Node.js para funcionar. Baixe a versão LTS (Long-Term Support):
 - a. <https://nodejs.org>

Preparação do ambiente no VSCode

Passo 1: Instalação das Ferramentas Base

1. **Instale o Visual Studio Code:** Use o instalador que você baixou. Aceite as opções padrão.
2. **Instale o Node.js:** Use o instalador baixado. Aceite as opções padrão.
3. **Instale o Gemini CLI:**
 - Abra o menu Iniciar e procure por "Command Prompt" ou "PowerShell" (no Windows) ou abra o "Terminal" (no macOS/Linux).
 - Digite o seguinte comando e pressione Enter:
npm install -g @google/gemini-cli
4. **Instale o plugin Gemini Code Assist:**
5. **Configure sua Chave de API do Gemini:**
 - Acesse o Google AI Studio:
<https://aistudio.google.com/app/apikey>
 - Clique em "**Create API key in new project**". Copie a chave gerada.
 - No terminal, execute o comando abaixo, substituindo **SUA_CHAVE_AQUI** pela chave que você copiou:
gemini auth SUA_CHAVE_AQUI
 - Alternativamente, você pode usar o GEMINI CLI apenas autenticando-se com Google

Preparação do ambiente no VSCode

Passo 2: Conversando com o Gemini para Montar o Ambiente

Agora, vamos usar o terminal para pedir ajuda ao Gemini. Esta é uma simulação de como você pode usar a ferramenta para pedir orientação e resolver dúvidas.

Aluno 🧑 : Abre o terminal e digite:

gemini "Olá, Gemini! Sou um estudante iniciante em Java. Acabei de instalar o VS Code e o seu CLI. O que mais eu preciso instalar no meu computador para criar e rodar um programa simples em Java que funcione no console usando os gerenciadores de pacotes Maven e Graddle? Me explique o que é cada coisa e por que eu preciso delas."

Siga os passos, com cuidado. Analise criticamente e questione se não entender ou quiser se aprofundar sobre algo. Se quiser agilizar o processo, pode fazer download do [maven](#) e do [graddle](#). O JDK, ele irá sugerir de acordo com o sistema operacional e contexto. Se for o windows, recomendo o [build do OpenJDK da Microsoft](#).

Preparação do ambiente no VSCode

Passo 3: Criando e Executando o "Olá, Mundo!" com Gemini

Vamos pedir ao Gemini o passo a passo para criar e rodar o projeto.

Aluno :

gemini "Agora, me dê os comandos exatos que eu preciso usar no terminal para criar um projeto Java 'Olá, Mundo!'. Depois, me diga como compilar e executar esse projeto."

Siga os passos, com cuidado. Analise criticamente e questione se não entender ou quiser se aprofundar sobre algo.

Vídeo de demonstração



Preparação do ambiente no IntelliJ IDEA

Passo 0: Downloads Prévios

1. **IntelliJ IDEA Community Edition:** A versão gratuita da IDE:
<https://www.jetbrains.com/idea/download>
2. **Node.js:** O Gemini CLI precisa do Node.js para funcionar. Baixe a versão LTS:
<https://nodejs.org>

Preparação do ambiente no IntelliJ IDEA

Passo 1: Instalação das Ferramentas Base

1. **Instale o IntelliJ IDEA:** Use o instalador que você baixou. Aceite as opções padrão.
2. **Instale o Node.js:** Use o instalador baixado. Aceite as opções padrão.
3. **Instale e configure o Gemini CLI:** Siga exatamente os **passos 1.3 e 1.4 do Roteiro 1** para instalar o CLI via **npm** e autenticar sua chave de API. O processo é o mesmo.

Preparação do ambiente no IntelliJ IDEA

Passo 2: Conversando com o Gemini para Montar o Ambiente

O diálogo com o Gemini será **exatamente o mesmo do Roteiro 1**. A necessidade de ter um JDK e o Maven é universal para o desenvolvimento Java, não importa a IDE.

Aluno : Abre o terminal e faz as mesmas perguntas do **Passo 2 do Roteiro 1** para descobrir que precisa instalar o **JDK** e o **Maven**.

Siga as instruções do Gemini para instalar o **JDK (Eclipse Temurin)** e o **Maven**.

Preparação do ambiente no IntelliJ IDEA

Passo 3: Criando e Executando o "Olá, Mundo!" no IntelliJ

Aqui, vamos usar a inteligência da IDE para nos ajudar, usando a informação que o Gemini nos deu.

Aluno : Pede ao Gemini o comando para criar um projeto Maven (mesma pergunta do **Passo 3 do Roteiro 1**). O Gemini vai responder com o comando `mvn archetype:generate ....`

Preparação do ambiente no IntelliJ IDEA

Agora, em vez de usar o terminal, faremos o seguinte no IntelliJ:

1. Crie um novo projeto:

- Abra o IntelliJ e clique em **New Project**.
- No menu à esquerda, selecione **Maven Archetype**.
- O IntelliJ provavelmente já terá detectado o JDK que você instalou. Se não, você pode adicioná-lo no campo "JDK".
- Clique na caixa **"Archetype"** e depois em **"Add Archetype..."**.
- Preencha com as informações que o Gemini nos daria para o comando:
 - GroupId: `org.apache.maven.archetypes`
 - ArtifactId: `maven-archetype-quickstart`
 - Version: (use a mais recente, ex: `1.4`)
- Clique em **Next**, dê um nome ao seu projeto (ex: `ola-mundo`) e clique em **Create**.

2. Explore o Projeto: O IntelliJ criará a mesma estrutura de pastas do Maven, com o arquivo `App.java` já contendo o "Hello World!".

3. Execute o Programa:

- Encontre o arquivo `App.java` em `src/main/java/br/com/meuprojeto/`.
- Você verá um ícone de **"play" verde** ao lado da linha `public static void main(String[] args)`.
- **Clique nesse ícone** e selecione **"Run 'App.main()'"**.

O IntelliJ irá compilar e executar o projeto, e você verá a saída "Hello World!" no painel "Run" na parte inferior da IDE. Ele fez o trabalho do `mvn compile` e `mvn exec:java` por trás dos panos, com um único clique.



Aula 03 - 03/03/2026

Revisão: sintaxe básica do Java, tipos de dados, laços de repetição.

Sumário

- Estrutura, Dados e Interação
 - Estrutura básica
 - Tipos de dados primitivos
 - Tipos por referência
 - Entrada de dados
 - Operadores aritméticos
- Lógica e Repetição
 - Operadores relacionais
 - Operadores lógicos
 - Estruturas condicionais
 - Laços de repetição

A Anatomia de um Programa e o Mundo dos Dados

- A estrutura `public class NomeDaClasse { ... }`.
- O método `main`: `public static void main(String[] args)`. O **ponto de entrada** de qualquer aplicação Java de console.
- O comando de saída: `System.out.println()`.
- Comentários de código (`//` e `/* ... */`).
- **Variáveis**: O que são (caixas na memória com um nome e um tipo).
- **Tipos de Dados Primitivos**:
 - `int`: para números inteiros (idade, quantidade).
 - `double`: para números de ponto flutuante (preço, altura, média).
 - `boolean`: para valores verdadeiros ou falsos (`true/false`).
 - `char`: para um único caractere ('A', '1', '!').
- **Tipo por Referência (introdução)**:
 - `String`: para textos. É um Objeto derivado de uma classe e não um tipo primitivo.

Atividade Prática Guiada

- Criar um novo projeto no VS Code.
- Desenvolver um programa chamado `MeuPerfil.java`.
- Declarar variáveis para guardar seu nome (`String`), idade (`int`), peso (`double`), se é doador de órgãos (`boolean`) e tipo sanguíneo (`char`).
- Inicializar essas variáveis e imprimi-las no console de forma organizada.

```
package br.com.meuprojeto;
```

```
public class MeuPerfil {  
    public static void main(String[] args) {  
        String nome = "Ricardo Pupo Larguesa";  
        int idade = 46;  
        double peso = 90.0;  
        boolean ehDoadorDeOrgaos = true;  
        char tipoSanguineo = 'O';  
        System.out.println("--- Meu Perfil Pessoal ---");  
        System.out.println("Nome Completo: " + nome);  
        System.out.println("Idade: " + idade + " anos");  
        System.out.println("Peso: " + peso + " kg");  
        System.out.println("É doador de órgãos? " + ehDoadorDeOrgaos);  
        System.out.println("Tipo Sanguíneo: " + tipoSanguineo);  
        System.out.println("-----");  
    }  
}
```

Desafio Rápido (5 min)

Adicionar mais duas variáveis

- altura
- se está empregado ou não

...e imprimir no console.

```
package br.com.meuprojeto;
```

```
public class MeuPerfil {  
    public static void main(String[] args) {  
        String nome = "Ricardo Pupo Larguesa";  
        int idade = 40;  
        double peso = 85.5;  
        boolean ehDoadorDeOrgaos = false;  
        char tipoSanguineo = 'O';  
        double altura = 1.75;  
        boolean estaEmpregado = true;  
        System.out.println("--- Meu Perfil Pessoal ---");  
        System.out.println("Nome Completo: " + nome);  
        System.out.println("Idade: " + idade + " anos");  
        System.out.println("Peso: " + peso + " kg");  
        System.out.println("Altura: " + altura + " m");  
        System.out.println("Está empregado? " + estaEmpregado);  
        System.out.println("É doador de órgãos? " + ehDoadorDeOrgaos);  
        System.out.println("Tipo Sanguíneo: " + tipoSanguineo);  
        System.out.println("-----");  
    }  
}
```

Tornando o Programa Interativo

- **Entrada de Dados:** Classe `Scanner`.
 - `import java.util.Scanner;` (import serve para importar pacotes).
 - `Scanner leitor = new Scanner(System.in);` (primeiro contato real com a criação de um Objeto!).
 - Métodos como `leitor.nextLine()` (para Strings), `leitor.nextInt()` (para inteiros) e `leitor.nextDouble()` (para doubles).
- **Operadores Aritméticos:** `+`, `-`, `*`, `/`, `%` (módulo/resto da divisão).
- **Conversão de Tipos (Type Casting):**
 - Na divisão de inteiros, `5 / 2` resulta em `2`
 - Para forçar uma conversão, basta colocar o tipo entre parênteses: `(double) 5 / 2`

Atividade Prática Guiada

- Desenvolver um programa chamado `CalculadoraDeIMC.java`.
- O programa deve usar o `Scanner` para pedir ao usuário seu nome, peso (em kg) e altura (em metros).
- Calcular o IMC (fórmula: `peso / (altura * altura)`).
- Exibir o resultado de forma amigável, como: "Olá, [Nome], seu IMC é [valor do IMC]".

Atenção para um bug comum: usar `nextInt()` e logo em seguida `nextLine()`, que pode "pular" a leitura da linha. Pode usar um `leitor.nextLine()` extra para consumir a quebra de linha.

```
package br.com.meuprojeto;
import java.util.Scanner;

public class CalculadoraDeIMC {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        System.out.println("--- Calculadora de Índice de Massa Corporal (IMC) ---");
        System.out.print("Por favor, digite o seu nome: ");
        String nome = leitor.nextLine();
        System.out.print("Agora, digite o seu peso em kg (ex: 90.0): ");
        double peso = leitor.nextDouble();
        System.out.print("Por fim, digite a sua altura em metros (ex: 1.8): ");
        double altura = leitor.nextDouble();
        double imc = peso / (altura * altura);
        System.out.println("-----");
        System.out.println("Olá, " + nome + "! Seu IMC calculado é: " + imc);
        System.out.println("-----");
        leitor.close();
    }
}
```

Desafio Rápido

Criar um novo programa para que ele calcule a média de duas notas digitadas pelo usuário.

```
package br.com.meuprojeto;  
import java.util.Scanner;
```

```
public class CalculadoraDeMedia {  
    public static void main(String[] args) {  
        Scanner leitor = new Scanner(System.in);  
        System.out.println("--- Calculadora de Média Simples ---");  
        System.out.print("Por favor, digite a primeira nota: ");  
        double nota1 = leitor.nextDouble();  
        System.out.print("Ótimo. Agora, digite a segunda nota: ");  
        double nota2 = leitor.nextDouble();  
        double media = (nota1 + nota2) / 2.0;  
        System.out.println("-----");  
        System.out.println("A média das notas é: " + media);  
        System.out.println("-----");  
        leitor.close();  
    }  
}
```

Lógica e Estruturas de Controle Condicional

- **Operadores Relacionais:** `==` (igual a), `!=` (diferente de), `>`, `<`, `>=`, `<=`.
- **Operadores Lógicos:** `&&` (E), `||` (OU), `!` (NÃO).
- **Estruturas Condicionais:**
 - `if`: "Se a condição for verdadeira, faça isso."
 - `else`: "Senão, faça aquilo."
 - `else if`: Para múltiplos testes condicionais.
- A comparação de Strings: Lembrar que `string1 == string2` NÃO funciona
 - Ao invés, use o método `string1.equals(string2)`.

Atividade Prática Guiada

- Evoluir a `CalculadoraDeIMC.java` ou o programa de média.
- Usando `if`, `else if` e `else`, exibir uma mensagem adicional baseada no resultado.
 - Para o IMC: "Abaixo do peso", "Peso normal", "Sobrepeso", etc.
 - Para a média: "Aprovado", "Recuperação", "Reprovado".

Monte a estrutura de `ifs` aninhados de forma lógica e clara, mostrando a importância da ordem das condições (ex: checar `media >= 7` antes de checar `media >= 5`).

```
package br.com.meuprojeto;
import java.util.Scanner;

public class DiagnosticoIMC {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        System.out.println("--- Diagnóstico de Índice de Massa Corporal (IMC) ---");
        System.out.print("Por favor, digite o seu nome: ");
        String nome = leitor.nextLine();
        System.out.print("Agora, digite o seu peso em kg (ex: 85.5): ");
        double peso = leitor.nextDouble();
        System.out.print("Por fim, digite a sua altura em metros (ex: 1.75): ");
        double altura = leitor.nextDouble();
        double imc = peso / (altura * altura);
        String classificacao;
        if (imc < 18.5) {
            classificacao = "Abaixo do peso";
        } else if (imc < 25) {
            classificacao = "Peso ideal (parabéns!)";
        } else if (imc < 30) {
            classificacao = "Levemente acima do peso (sobrepeso)";
        } else if (imc < 35) {
            classificacao = "Obesidade Grau I";
        } else if (imc < 40) {
            classificacao = "Obesidade Grau II (severa)";
        } else {
            classificacao = "Obesidade Grau III (mórbida)";
        }
        System.out.println("-----");
        System.out.printf("Olá, %s! Seu IMC calculado é: %.2f%n", nome, imc);
        System.out.println("Diagnóstico: " + classificacao); // Imprimimos a variável que foi preenchida pelo if/else.
        System.out.println("-----");
        leitor.close();
    }
}
```

Desafio Rápido

Criar um programa que pede a idade do usuário e informa se ele é "menor de idade", "adulto" ou "idoso" (acima de 65 anos).


```
package br.com.meuprojeto;
```

```
import java.util.Scanner;
```

```
public class VerificadorDeldade {
```

```
    public static void main(String[] args) {
```

```
        Scanner leitor = new Scanner(System.in);
```

```
        System.out.println("--- Verificador de Faixa Etária ---");
```

```
        System.out.print("Por favor, digite a sua idade: ");
```

```
        int idade = leitor.nextInt();
```

```
        String faixaEtaria;
```

```
        if (idade < 0) {
```

```
            faixaEtaria = "Idade inválida!";
```

```
        } else if (idade < 18) {
```

```
            faixaEtaria = "Menor de idade";
```

```
        } else if (idade <= 65) {
```

```
            faixaEtaria = "Adulto";
```

```
        } else {
```

```
            faixaEtaria = "Idoso";
```

```
        }
```

```
        System.out.println("-----");
```

```
        System.out.println("Classificação: " + faixaEtaria);
```

```
        System.out.println("-----");
```

```
        leitor.close();
```

```
    }
```

```
}
```

O Poder da Repetição

- **Laços de Repetição (Loops):** Para que servem? Evitar a repetição de código.
- **Laço `for`:** Ideal para quando se sabe o número de repetições.
 - Sintaxe: `for (inicialização; condição; incremento) { ... }`.
Detalhar cada parte.
- **Laço `while`:** Ideal para quando a repetição depende de uma condição que pode mudar.
 - Sintaxe: `while (condição) { ... }`.

Evite loops infinitos, especialmente no `while`, com uma condição de parada clara.

Atividade Prática Guiada

- **Exemplo com `for`:** Criar um programa `Tabuada.java` que pede um número ao usuário e imprime a tabuada de multiplicação desse número, de 1 a 10.
- **Exemplo com `while`:** Criar um pequeno jogo `AdivinheONumero.java`. O programa define um número qualquer (ex: `int numeroSecreto = 7;`) ou usando `Math.random()`. O usuário digita palpites até acertar. A cada erro, o programa diz se o palpite foi maior ou menor que o número secreto. O loop continua `while (palpite != numeroSecreto)`.

```
package br.com.meuprojeto;
import java.util.Scanner;

public class Tabuada {

    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        System.out.println("--- Gerador de Tabuada ---");
        System.out.print("Digite o número para o qual você deseja a tabuada: ");
        int numero = leitor.nextInt();
        System.out.println("-----");
        System.out.println("Tabuada de " + numero + ":");
        for (int i = 1; i <= 10; i++) {
            int resultado = numero * i;
            System.out.println(numero + " x " + i + " = " + resultado);
        }
        System.out.println("-----");
        leitor.close();
    }
}
```

```
package br.com.meuprojeto;
import java.util.Random;
import java.util.Scanner;

public class AdivinheONumero {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        Random gerador = new Random();
        int numeroSecreto = gerador.nextInt(100) + 1;
        int palpite = 0;
        int tentativas = 0;
        System.out.println("--- Jogo de Adivinhação ---");
        System.out.println("Tente adivinhar o número que eu escolhi, entre 1 e 100!");
        while (palpite != numeroSecreto) {
            System.out.print("Digite o seu palpite: ");
            palpite = leitor.nextInt();
            tentativas++; // Incrementa o número de tentativas
            if (palpite == numeroSecreto) {
                System.out.println("*****");
                System.out.println("Parabéns, você acertou em " + tentativas + " tentativas!");
                System.out.println("*****");
            } else if (palpite < numeroSecreto) {
                System.out.println("O número secreto é MAIOR que " + palpite);
            } else {
                System.out.println("O número secreto é MENOR que " + palpite);
            }
        }
        System.out.println("Fim de jogo!");
        leitor.close();
    }
}
```

Desafio

Modificar o programa `Tabuada.java` para que ele pergunte ao usuário se deseja calcular a tabuada de outro número ao final da execução. O programa principal deve ficar dentro de um loop `do-while` que continua enquanto a resposta do usuário for "sim".

```
package br.com.meuprojeto;
```

```
import java.util.Scanner;
```

```
public class TabuadaInterativa {
```

```
    public static void main(String[] args) {
```

```
        Scanner leitor = new Scanner(System.in);
```

```
        String resposta;
```

```
        do {
```

```
            System.out.println("\n--- Gerador de Tabuada ---");
```

```
            System.out.print("Digite o número para o qual você deseja a tabuada: ");
```

```
            int numero = leitor.nextInt();
```

```
            System.out.println("-----");
```

```
            System.out.println("Tabuada de " + numero + ":");
```

```
            for (int i = 1; i <= 10; i++) {
```

```
                int resultado = numero * i;
```

```
                System.out.println(numero + " x " + i + " = " + resultado);
```

```
            }
```

```
            System.out.println("-----");
```

```
            System.out.print("Deseja calcular outra tabuada? (digite 'sim' para continuar): ");
```

```
            resposta = leitor.next(); // Lemos a resposta do usuário.
```

```
        } while (resposta.equalsIgnoreCase("sim"));
```

```
        System.out.println("\nObrigado por usar o gerador de tabuadas. Até a próxima!");
```

```
        leitor.close();
```

```
    }
```

```
}
```

Tarefa Final : Simulador de Caixa Eletrônico

Objetivo: O objetivo desta tarefa é aplicar e consolidar os conceitos fundamentais da linguagem Java aprendidos em aula. Você irá praticar:

- A declaração e manipulação de variáveis.
- A recepção de dados do usuário via `Scanner`.
- O controle de fluxo do programa com `if`, `else if` e `else`.
- A criação de um menu interativo com um laço de repetição (`while` ou `do-while`).

Descrição do Projeto: Você deverá criar um programa em Java que simule as operações básicas de um caixa eletrônico. O programa deve ser executado no console e apresentar um menu de opções para o usuário. A aplicação continuará rodando e exibindo o menu até que o usuário escolha a opção "Sair".

Estrutura do Código (Dicas):

- Use um laço `while` ou `do-while` para manter o menu ativo. A condição de parada do laço será quando a opção do usuário for `4`.
- Dentro do laço, use uma estrutura `if-else if-else` ou, melhor ainda, um `switch` para tratar a opção escolhida pelo usuário.
- Não se esqueça de declarar a variável `saldo` fora do loop, caso contrário, seu valor será resetado a cada nova iteração!

Requisitos Funcionais:

1. **Saldo Inicial:** O programa deve começar com uma variável `double saldo` inicializada com um valor pré-definido (ex: R\$ 2.500,00).
2. **Menu de Opções:** A cada iteração, o programa deve limpar a tela (opcional) e exibir um menu com as opções:
 - **Opção 1 - Consultar Saldo:** Se o usuário digitar `1`, o programa deve exibir o valor atual da variável `saldo`.
 - **Opção 2 - Receber Valor (Depósito):** Se o usuário digitar `2`, o programa deve perguntar qual valor ele deseja depositar. Em seguida, o valor informado deve ser somado ao `saldo`.
 - **Opção 3 - Transferir Valor (Saque):** Se o usuário digitar `3`, o programa deve perguntar qual valor ele deseja sacar. Antes de efetuar o saque, o programa deve verificar se o valor a ser sacado é menor ou igual ao `saldo` disponível.
 - i. Se o saldo for suficiente, o valor do saque deve ser subtraído do `saldo`.
 - ii. Se o saldo for insuficiente, o programa deve exibir uma mensagem de "Saldo insuficiente".
 - **Opção 4 - Sair:** Se o usuário digitar `4`, o programa deve exibir uma mensagem de despedida e encerrar a execução.
 - **Opção Inválida:** Se o usuário digitar qualquer outra coisa, o programa deve exibir uma mensagem de "Opção inválida" e mostrar o menu novamente.



Aula 04 - 10/03/2026

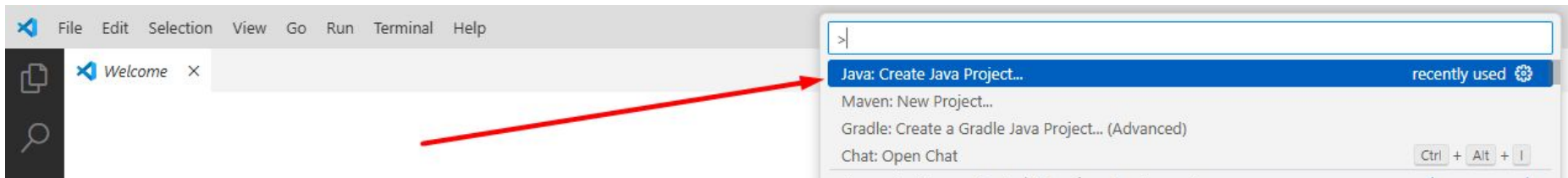
Classes, Atributos e Métodos

Sumário

- Classes
 - Declarando classes
 - Contextualizando variáveis e funções
 - Instanciando objetos
 - Atributos
 - Métodos
 - Encapsulamento

Criando um projeto

Digite **CTRL+SHIFT+P** e procure a opção de criar um projeto Java, escolha uma pasta e dê o nome do projeto, que será usado para criar uma nova pasta



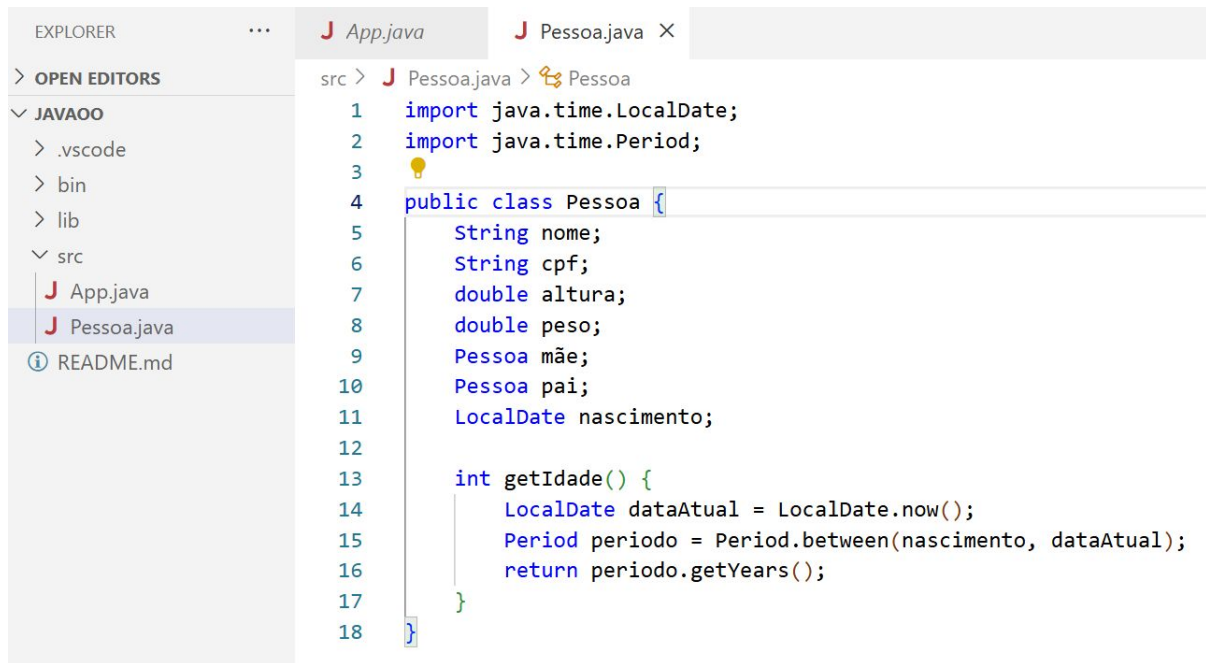
Declarando classes

Crie um novo arquivo chamado “Pessoa.java”. Repare na palavra reservada “class” seguida de um nome e um par de chaves para delimitar escopo



Contextualizando estado e comportamento

Variáveis e funções podem ser declaradas dentro do escopo da classe, especificando para ela **ESTADO** e **COMPORTAMENTO**



The screenshot shows a code editor with two tabs: 'App.java' and 'Pessoa.java'. The 'Pessoa.java' tab is active, displaying the following code:

```
src > J Pessoa.java > Pessoa
1  import java.time.LocalDate;
2  import java.time.Period;
3
4  public class Pessoa {
5      String nome;
6      String cpf;
7      double altura;
8      double peso;
9      Pessoa mãe;
10     Pessoa pai;
11     LocalDate nascimento;
12
13     int getIdade() {
14         LocalDate dataAtual = LocalDate.now();
15         Period periodo = Period.between(nascimento, dataAtual);
16         return periodo.getYears();
17     }
18 }
```

The Explorer panel on the left shows the project structure with folders '.vscode', 'bin', 'lib', and 'src'. The 'src' folder contains 'App.java', 'Pessoa.java', and 'README.md'. The 'Pessoa.java' file is selected.

Implementando um método principal

O “main” torna a classe executável

Usamos o new para instanciar objetos de uma classe



```
J App.java  J Pessoa.java X
src > J Pessoa.java > Pessoa > main(String[])
4  public class Pessoa {
13     int getIdade() {
17 }
18 public static void main(String[] args) {
19     Pessoa pessoa = new Pessoa();
20     pessoa.nome = "Ricard";
21     pessoa.cpf = "123.456.789-00";
22     pessoa.altura = 1.8;
23     pessoa.peso = 90.0;
24     pessoa.nascimento = LocalDate.of(year:1979, month:7, dayOfMonth:1);
25     System.out.println("Idade: " + pessoa.getIdade());
26 }
27 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\rlarg\OneDrive\Documentos\Java\Java00> c:: cd 'c:\Users\rlarg\OneDrive\
C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMes
g\OneDrive\Documentos\Java\Java00\bin' 'Pessoa'
Idade: 46
○ PS C:\Users\rlarg\OneDrive\Documentos\Java\Java00> 
```

Atributos

São variáveis declaradas dentro da classe e, por isso, encapsuladas em um contexto de objeto

- A declaração possui os seguintes componentes:
 - Modificador de acesso:
 - Serve para definir em que classes o atributo pode ser acessado
 - Por exemplo, `private` ou `public`
 - Tipo de dado:
 - `int`, `float`, `double`, `String`, etc.
 - Nome do atributo
 - Segue as mesmas regras de identificação de variáveis

Tarefa 01

No projeto:

- Crie um pacote chamado “br.gov.sp.fatec.pg.oo”
- Crie uma classe chamada “Data” com os seguintes atributos:
 - dia, do tipo inteiro
 - mês, do tipo inteiro
 - ano, do tipo inteiro

EXPLORER

...

J App.java

J Pessoa.java

J Data.java

×

> OPEN EDITORS

✓ JAVA00

> .vscode

> bin

> lib

✓ src\br\gov\sp\fattec\pg\oo

J App.java

J Data.java

J Pessoa.java

src > br > gov > sp > fatec > pg > oo > J Data.java > Data

1 package br.gov.sp.fatec.pg.oo;

2

3 public class Data {

4 int dia;

5 int mês;

6 int ano;

7 }

Tarefa 02

No mesmo projeto:

- Na classe “App”, implemente um método principal que:
 - Instancie um objeto chamado “hoje” da classe Data com os atributos de hoje
 - Instancie outro objeto chamado “meuNascimento” da classe Data com os atributos da data do seu nascimento

EXPLORER

...

> OPEN EDITORS

▼ JAVA00

> .vscode

> bin

> lib

▼ src\br\gov\sp\ fatec\pg\oo

J App.java

J Data.java

J Pessoa.java

i README.md

J App.java X

J Pessoa.java

J Data.java

src > br > gov > sp > fatec > pg > oo > J App.java > App > main(String[])

```
1  package br.gov.sp.fatec.pg.oo;
2  public class App {
    Run | Debug
3      public static void main(String[] args) throws Exception {
4          Data hoje = new Data();
5          hoje.dia = 1;
6          hoje.mês = 9;
7          hoje.ano = 2025;
8
9          Data meuNascimento = new Data();
10         meuNascimento.dia = 1;
11         meuNascimento.mês = 7;
12         meuNascimento.ano = 1979;
13     }
14 }
```

Tarefa 03

No mesmo projeto

- No mesmo pacote

- Crie uma classe chamada Horário com os seguintes atributos:
 - horas, do tipo inteiro
 - minutos, do tipo inteiro
 - segundos, do tipo inteiro
- Altere o método principal da classe Main para:
 - Instanciar um objeto chamado “agora” da classe Horário com os atributos do horário atual
 - Instanciar um objeto chamado “intervalo” da classe Horário com os atributos do horário do intervalo

EXPLORER

...

▼ JAVA1

> .vscode

> bin

> lib

▼ src\br\gov\sp\fattec\pg\oo

J App.java

J Horário.java

i README.md

J Horário.java X

J App.java

src > br > gov > sp > fatec > pg > oo > J Horário.java > 📁 Horário

1 package br.gov.sp.fatec.pg.oo;

2

3 public class Horário {

4 int horas;

5 int minutos;

6 💡 int segundos;

7

}

EXPLORER

...

> OPEN EDITORS

▼ JAVA00

> .vscode

> bin

> lib

▼ src\br\gov\sp\ fatec\pg\oo

J App.java

J Data.java

J Horario.java

J Pessoa.java

(i) README.md

J App.java X

J Horario.java

J Pessoa.java

J Data.java

src > br > gov > sp > fatec > pg > oo > J App.java > App > main(String[])

```
1  package br.gov.sp.fatec.pg.oo;
2  public class App {
    Run | Debug
3      public static void main(String[] args) throws Exception {
4          Horario agora = new Horario();
5          agora.horas = 14;
6          agora.minutos = 45;
7          agora.segundos = 30;
8
9          Horario intervalo = new Horario();
10         intervalo.horas = 15;
11         intervalo.minutos = 00;
12         intervalo.segundos = 00;
13     }
```

Encapsulamento

Pode-se definir um modificador de acesso ao atributo, tornando-o:

- public
 - Acessível por todas as classes do projeto
- private
 - Acessível apenas dentro da própria classe

Por que faríamos isso?

Para proteger o acesso
aos atributos,
garantindo o
comportamento
desejado de uma
classe

src > br > gov > sp > fatec > pg > oo >  Data.java >  Data >  setDia(int)

```
1  package br.gov.sp.fatec.pg.oo;
2
3  public class Data {
4      private int dia;
5      private int mês;
6      private int ano;
7
8      public void setDia(int novoDia) {
9          if(novoDia < 1) dia = 1;
10         else if(novoDia > 31) dia = 31;
11         else dia = novoDia;
12     }
13
14     public int getDia() {
15         return dia;
16     }
17 }
```

Mas impacta na atribuição

J App.java X

J Horário.java

J Pessoa.java

J Data.java ●

src > br > gov > sp > fatec > pg > oo > J App.java > App > main(String[])

```
1 package br.gov.sp.fatec.pg.oo;
2 public class App {
    Run | Debug
3     public static void main(String[] args) throws Exception {
4         Data hoje = new Data();
5         hoje.setDia(novoDia:1);
6         hoje.setMês(mês:9);
7         hoje.setAno(ano:2025);
8
9         Data meuNascimento = new Data();
10        meuNascimento.setDia(novoDia:1);
11        meuNascimento.setMês(mês:7);
12        meuNascimento.setAno(ano:1979);
13    }
```

Encapsulamento em Java

Os atributos são convencionalmente declarados como private e encapsulados em métodos público de acesso:

- Leitura (método get)
 - Retorna o tipo de dado do atributo
 - Não recebe parâmetro
- Gravação (método set)
 - Retorna void
 - Recebe um parâmetro de mesmo tipo para ser atribuído

src > J Teste.java > Teste

```
1  public class Teste {  
2      private int valor;  
3  
4      public int getValor() {  
5          return valor;  
6      }  
7  
8      public void setValor(int valor) {  
9          this.valor = valor;  
10     }  
11 }
```

Tarefa 04

No mesmo projeto

- No mesmo pacote
 - Encapsule os atributos da classe “Data”
 - Encapsule os atributos da classe “Horário”
 - Ajuste as atribuições da classe “App”
 - Os 4 objetos, 2 de Data e 2 de Horário

PARA FIXAR: Simulador de Caixa Eletrônico OO

Objetivo: Reescrever a tarefa da aula anterior utilizando orientação a objetos:

- Crie a classe **Conta** com um atributo para o **saldo** e os métodos:
 - double **consultar**()
 - void **depositar**(double valor)
 - void **sacar**(double valor)
- Crie a classe **App** com o método **main** instanciando um objeto da classe **Conta** e **implementando a interface de menu** para manipular o saldo



Aula 05 - 17/03/2026

Objetos, instâncias e construtores

Atributos de instância

Existem no **contexto de uma instância** e só podem ser acessados através de um **objeto instanciado**:

- São os atributos que vimos até agora

Exemplos

- **Classe Aluno**
 - String **matricula**: Cada aluno tem um número de matrícula único.
 - String **nome**: O nome é específico de cada aluno.
 - double[] **notas**: As notas de um aluno não afetam as notas de outro.
- **Classe Produto (em um e-commerce)**
 - String **sku**: O código identificador de um produto é único.
 - double **preco**: O preço pode variar para produtos diferentes, mesmo que sejam da mesma categoria.
 - int **quantidadeEmEstoque**: A quantidade disponível é uma característica do produto específico, não de todos os produtos da loja.
- **Classe ContaBancaria**
 - String **numeroDaConta**: Cada conta tem um número exclusivo.
 - double **saldo**: O saldo é a característica mais individual de uma conta.
 - Cliente **titular**: O dono de uma conta é específico para aquela instância.
- **Classe Postagem (em uma rede social como o X ou Instagram)**
 - String **textoConteudo**: O texto de uma postagem é único.
 - Usuario **autor**: Cada postagem é feita por um autor específico.
 - LocalDateTime **dataDePublicacao**: O momento exato da publicação pertence àquela postagem.
 - int **numeroDeCurtidas**: A contagem de "likes" é individual para cada postagem.
- **Classe ConexaoBancoDeDados**
 - String **sessionId**: O ID da sessão é único para cada conexão estabelecida.
 - long **tempoInicioConexao**: O momento em que a conexão foi aberta é específico dela.

Atributos de classe

São atributos de atribuição estática, e o valor atribuído está no contexto da classe

- Usamos 'static' na declaração
- É acessado através da classe e sua atribuição independe de parâmetros

Exemplos

- Classe Configuracao

- `public static final String URL_API = "https://api.meusistema.com/v2/"`: Uma constante que define a URL base da API para toda a aplicação. Não faz sentido cada objeto ter uma cópia disso.
- `public static final int TIMEOUT_PADRAO = 5000`: O tempo máximo de espera padrão para requisições, compartilhado por todos os componentes que o utilizam.

- Classe Usuario

- `private static int contadorDeInstancias = 0`: Um contador que é incrementado no construtor para saber quantos objetos Usuario foram criados no total durante a execução do sistema.
- `public static final int TAMANHO_MINIMO_SENHA = 8`: Uma regra de negócio que se aplica a todos os usuários, sem exceção.

- Classe Calculadora

- `public static final double PI = 3.14159265`: Constantes matemáticas são o exemplo clássico. O valor de PI é universal e não pertence a uma "instância" de cálculo.

- Classe Logger (Padrão Singleton)

- `private static Logger instanciaUnica`: No padrão Singleton, a única instância da classe é armazenada em uma variável estática para garantir que todos os pontos do código usem o mesmo objeto de log.

- Classe Produto

- `public static double IMPOSTO_FEDERAL_PADRAO = 0.0925`: Se uma alíquota de imposto é a mesma para todos os produtos cadastrados, ela pode ser um atributo de classe. Se o imposto variar por produto, aí seria um atributo de instância.

Exemplo

```
1  package br.gov.sp.fatec.pg.oo;
2
3  public class Carro {
4
5      public static int numeroDeRodas = 4;
6
7      private String cor;
8      private String placa;
9
10     public void exibirInformacoes() {
11         // Acessando atributos de instância: Apenas pelo 'this'
12         System.out.println(x:"-----");
13         System.out.println("Carro com placa: " + this.placa);
14         System.out.println("Cor do carro (instância): " + this.cor);
15         // Acessando atributo de classe: via 'this' ou diretamente pela classe.
16         System.out.println("Número de rodas (classe): " + Carro.numeroDeRodas);
17         System.out.println(x:"-----\n");
18     }
19 }
```

Para que mais serviria isso?

```
3 public class Data {  
4     private int dia;  
5     private int mês;  
6     private int ano;  
7  
8     private static String[] meses = {"JAN", "FEV", "MAR"  
9                                     , "ABR", "MAI", "JUN"  
10                                    , "JUL", "AGO", "SET"  
11                                    , "OUT", "NOV", "DEZ"};  
12  
13     public String getNomeDoMês() {  
14         return meses[this.mês-1];  
15     }
```

Método

São funções declaradas dentro da classe e, por isso, encapsuladas em um contexto de objeto

- A declaração de um método possui cinco componentes:
 - **Modificador:** private, public ou outros que conheceremos mais tarde
 - **Tipo de retorno:** Tipo de dado retornado pelo método ou void caso seja procedural
 - **Nome do método:** Respeitando as mesmas regras de identificação de variáveis
 - **Lista de parâmetros:** Separados por ',' e entre parênteses, apresenta as variáveis do contexto de execução do método
 - **Corpo do método:** Delimitado pelo par de chaves '{' e '}', armazena o código executado pelo método

Exemplo

```
5 public class Pessoa {
6     String nome;
7     String cpf;
8     double altura;
9     double peso;
10    Pessoa mãe;
11    Pessoa pai;
12    LocalDate nascimento;
13
14    int getIdade() {
15        LocalDate dataAtual = LocalDate.now();
16        Period periodo = Period.between(nascimento, dataAtual);
17        return periodo.getYears();
18    }
19    public static void main(String[] args) {
```

Tarefa 01

- No projeto da aula anterior
 - Crie na classe Data um método chamado “getData” sem parâmetros que retorne uma String com a data no formato “DD/MM/AAAA”

Solução

```
public String getData() {  
    String out = "";  
    if (getDia() < 10) out += "0";  
    out += getDia() + "/";  
    if (getMês() < 10) out += "0";  
    out += getMês() + "/";  
    out += getAno();  
    return out;  
}
```

Tarefa 02

- No mesmo projeto:
 - Crie na classe Horário um método chamado “getHorário” sem parâmetros que retorne uma String com o horário no formato “HH:MM:SS”

Solução

```
public String getHorário() {  
    String out = "";  
    if(horas < 10) out += "0";  
    out += horas+":";  
    if(minutos < 10) out += "0";  
    out += minutos+":";  
    if(segundos < 10) out += "0";  
    out += segundos;  
    return out;  
}
```

Métodos construtores

Métodos especiais que são executados EXCLUSIVAMENTE quando um objeto é instanciado

- São declarados com o mesmo nome da classe, sem tipo de dados de retorno
- Podem ser sobrecarregados
- A execução ocorre quando executado um comando “new”

Exemplo

```
3  public class Data {  
4      private int dia;  
5      private int mês;  
6      private int ano;  
7  
8      public Data(){  
9          dia = 1;  
10         mês = 1;  
11         ano = 1970;  
12     }  
13  
14     public Data(int dia, int mês, int ano){  
15         this.dia = dia;  
16         this.mês = mês;  
17         this.ano = ano;  
18     }  
19 }
```

Tarefa 03

- No mesmo projeto:
 - Crie na classe Horário:
 - Um construtor sem parâmetros que inicialize o objeto com 00:00:00
 - Um construtor parametrizado que inicialize o objeto com os valores de horas, min e seg informados nos parâmetros
 - Use os métodos set ao invés de atribuir direto

Solução

```
3 public class Horario {
4     private int horas;
5     private int minutos;
6     private int segundos;
7
8     public Horario() {
9         horas=minutos=segundos = 0;
10    }
11    public Horario(int h, int m, int s) {
12        setHoras(h);
13        setMinutos(m);
14        setSegundos(s);
15    }
```

Tarefa 04

- No mesmo projeto:
 - Crie na classe Data:
 - Crie um método chamado isBissexto() que retorne true se o ano da data é bissexto ou false caso contrário
 - Crie na classe Horário:
 - Crie um método chamado isManhã() que retorne true se o horário está entre 6h e 12h e false caso contrário
 - Crie um método chamado isTarde() que retorne true se o horário está entre 12h e 18h e false caso contrário
 - Crie um método chamado isNoite() que retorne true se o horário está entre 18h e 0h e false caso contrário
 - Crie um método chamado isMadrugada() que retorne true se o horário está entre 0h e 6h e false caso contrário

Solução 1

```
69  [- public boolean isBisexto() {
70      if(ano % 400 == 0)
71          return true;
72      I else{
73          if(ano % 100 == 0)
74              return false;
75      76      else if(ano % 4 == 0)
77          return true;
78      79      else
80          return false;
      }
```

Solução 2

```
57 public boolean isMadrugada() {  
    if(horas >= 0 && horas < 6) return true;  
    else return false;  
60 }  
61 public boolean isManhã() {  
    if(horas >= 6 && horas < 12) return true;  
    else return false;  
64 }  
65 public boolean isTarde() {  
    if(horas >= 12 && horas < 18) return true;  
    else return false;  
68 }  
69 public boolean isNoite() {  
    if(horas >= 18 && horas < 24) return true;  
    else return false;  
71 }  
72 }
```

PARA FIXAR: QUIZ

Objetivo: Criar um projeto de aplicação que apresente um QUIZ (tema livre) com 10 questões:

- Os dados devem ser gravados em objetos de memória como ArrayList ou HashMap
- Ao final do QUIZ, deve ser apresentada a média de acertos



Aula 06 - 24/03/2026

Herança, polimorfismo, classes abstratas e interfaces

Sumário

- Herança: Reutilização e Especialização
 - Conceito de Superclasse e Subclasse (extends)
 - A chamada super() em construtores
- Polimorfismo: Múltiplas Formas, Um Único Método
 - Sobrescrita de Métodos (@Override)
- Classes Abstratas: O Molde Incompleto
 - Quando e por que usar? Métodos abstratos
- Interfaces: O Contrato de Comportamento
 - Diferenças cruciais e implementação múltipla
- Design Patterns em Ação
 - Strategy (com Interfaces)
 - Template Method (com Classes Abstratas)
- Exercícios Práticos

Herança: Reutilização e Especialização

- **O que é?**

- Herança é o pilar da reutilização. Permite que uma classe (filha ou subclasse) herde atributos e métodos de outra classe (mãe ou superclasse).

- **Sintaxe:**

- Usamos a palavra-chave `extends`.

- **Java**

```
public class Gerente extends Funcionario {  
    // Gerente herda tudo de Funcionario  
}
```

- **Para que serve?**

- **Generalização/Especialização:** Criar uma hierarquia onde classes mais específicas (como Gerente) especializam uma classe mais genérica (como Funcionario).
- **Evitar Duplicação de Código:** Atributos como nome e cpf não precisam ser reescritos.

A Chamada `super()`

- **Construtores não são herdados!**

- Quando um objeto da subclasse é criado, o construtor da superclasse precisa ser chamado primeiro para inicializar os atributos herdados.

- **Como fazer?**

- Use `super()` para invocar o construtor da classe mãe. Deve ser a primeira linha do construtor da subclasse.

Exemplo

J Funcionario.java

src > oo > J Funcionario.java > Funcionario

```
1 package oo;
2
3 public class Funcionario {
4     protected String nome;
5     protected double salario;
6
7     public Funcionario(String nome, double salario) {
8         this.nome = nome;
9         this.salario = salario;
10    }
11 }
```

J Gerente.java 1

src > oo > J Gerente.java > Gerente

```
1 package oo;
2
3 public class Gerente extends Funcionario {
4     private String area;
5
6     public Gerente(String nome, double salario, String area) {
7         super(nome, salario); // Chama o construtor de Funcionario
8         this.area = area;
9     }
10 }
```

Polimorfismo: Múltiplas Formas

- **O que é?**

- Do grego, "muitas formas". É a capacidade de um objeto ser tratado como uma instância de sua própria classe, de sua superclasse ou de uma interface que implementa.

- **A Mágica da Sobrescrita (@Override)**

- O polimorfismo brilha quando sobrescrevemos métodos. A JVM decide, em tempo de execução, qual implementação do método chamar, com base no tipo real do objeto.

Exemplo

J Funcionario.java

src > oo > J Funcionario.java > Funcionario > getBonificacao()

```
1 package oo;
2
3 public class Funcionario {
4     protected String nome;
5     protected double salario;
6
7     public Funcionario(String nome, double salario) {
8         this.nome = nome;
9         this.salario = salario;
10    }
11
12    public double getBonificacao() {
13        return this.salario * 0.10;
14    }
15 }
```

J Gerente.java 1 X

src > oo > J Gerente.java > Gerente

```
1 package oo;
2
3 public class Gerente extends Funcionario {
4     private String area;
5
6     public Gerente(String nome, double salario, String area) {
7         super(nome, salario); // Chama o construtor de Funcionario
8         this.area = area;
9     }
10    @Override // Boa prática para garantir a sobrescrita
11    public double getBonificacao() {
12        return this.salario * 0.15 + 1000;
13        /* No main:
14         Funcionario f = new Gerente("Ricardo", 10000.0, "TI");
15         System.out.println(f.getBonificacao());
16        */
17    }
18 }
```

Classes Abstratas: O Molde Incompleto

- **O que são?**

- Classes que não podem ser instanciadas. Servem apenas como um modelo (molde) para outras classes.

- **Quando usar?**

- Quando você tem um conceito genérico que não faz sentido existir sozinho. Ex: "Veículo" é um conceito, mas você dirige um "Carro" ou uma "Moto".
- Para garantir que subclasses implementem certos métodos.

- **Métodos Abstratos:**

- São métodos declarados sem corpo ({}). Obrigam as subclasses concretas a fornecerem uma implementação.

Exemplo

J Animal.java X

src > oo > J Animal.java >  Animal

```
1  package oo;
2
3  public abstract class Animal {
4      // Método concreto, herdado por todos
5      public void respirar() {
6          System.out.println(x:"Respirando...");
7      }
8
9      // Método abstrato, deve ser implementado pelas subclasses
10      public abstract void fazerBarulho();
11 }
```

Interfaces: O Contrato de Comportamento

- **O que são?**

- Um "contrato" 100% abstrato que define um conjunto de comportamentos (métodos) que uma classe deve ter. Uma classe implements uma interface.

- **Regras importantes:**

- Uma classe só pode extends uma outra classe.
- Uma classe pode implements várias interfaces (herança múltipla de tipos).
- São ideais para conectar partes do sistema que não têm uma relação de herança direta.

Exemplo

J Autenticavel.java ●

src > oo > J Autenticavel.java > Autenticavel

```
1 package oo;
2
3 public interface Autenticavel {
4     boolean autenticar(String senha);
5 }
```

J Gerente.java 1 ●

src > oo > J Gerente.java > Gerente > autenticar(String)

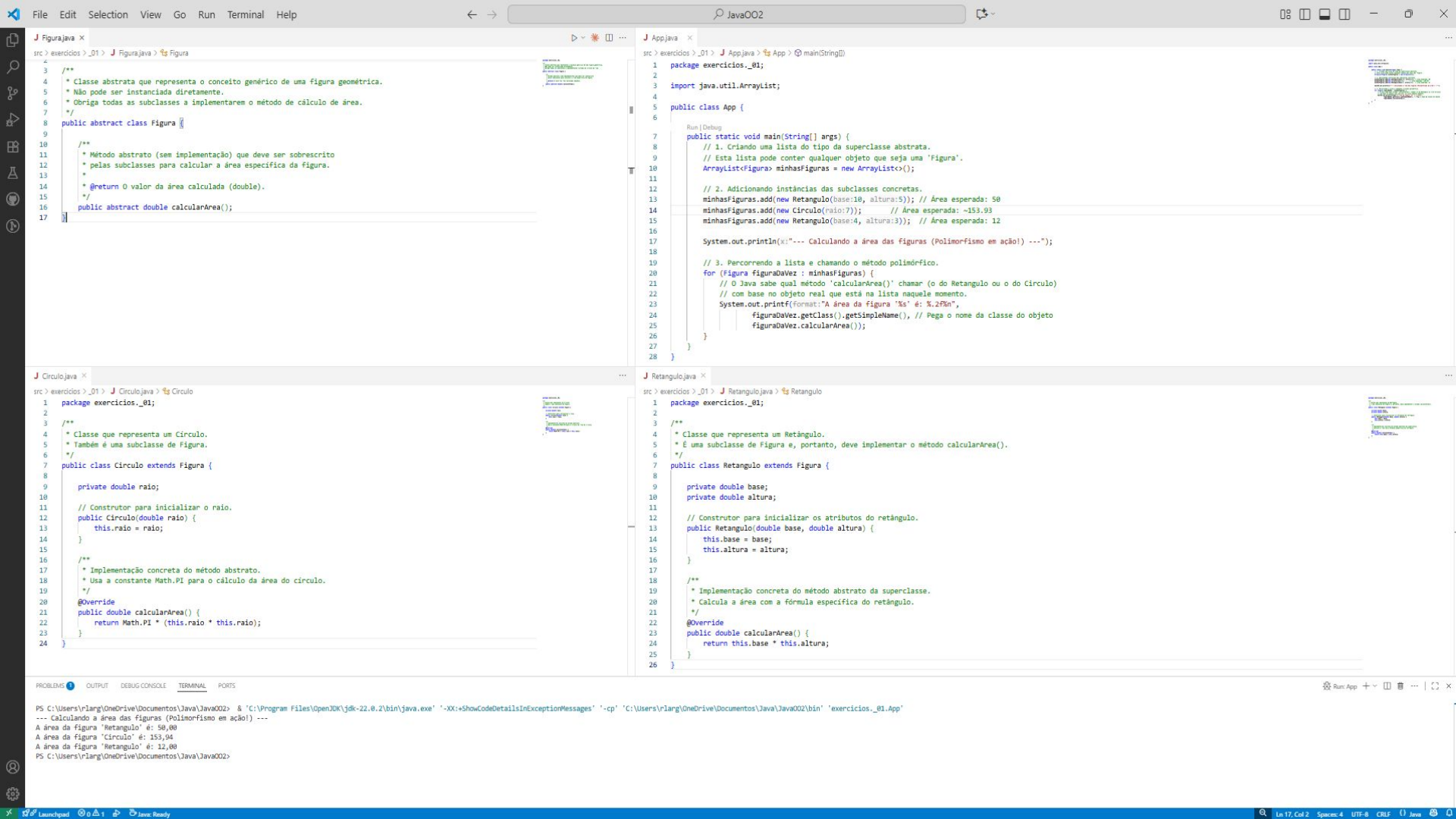
```
1 package oo;
2
3 public class Gerente extends Funcionario implements Autenticavel {
4     private String area;
5
6 > public Gerente(String nome, double salario, String area) { ...
10 @Override // Boa prática para garantir a sobrescrita
11 > public double getBonificacao() { ...
18
19     public boolean autenticar(String senha) {
20         return "1234".equals(senha);
21     }
22 }
```

Exercício Prático 01: Figuras Geométricas

Objetivo: Aplicar herança, polimorfismo e classes abstratas.

Enunciado:

1. Crie uma classe abstrata `Figura` com um método abstrato `public abstract double calcularArea();`.
2. Crie duas subclasses concretas: `Retangulo` (com atributos `base` e `altura`) e `Circulo` (com atributo `raio`).
3. Implemente o método `calcularArea()` em cada subclasse com a fórmula correta.
4. Na classe `App`, crie um `ArrayList` de `Figura`. Adicione instâncias de `Retangulo` e `Circulo` a ele.
5. Percorra a lista com um laço e, para cada figura, imprima o nome da classe e o valor da sua área, sem precisar saber o tipo específico da figura no laço.



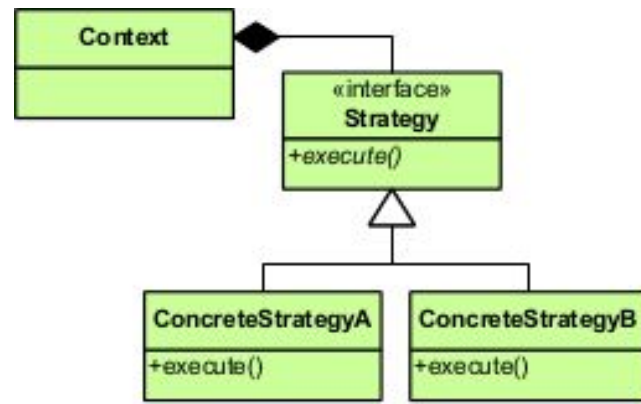
Aplicando Interfaces: Pattern "Strategy"

Problema: Como calcular diferentes impostos sobre um produto sem usar um `if-else` gigante e permitir adicionar novos impostos facilmente?

Solução com Strategy:

1. **Definir a Estratégia:** Crie uma interface chamada `Imposto` com um método `double calcula(double valor)`.
2. **Criar as Estratégias Concretas:** Crie classes `ICMS`, `IPI`, `PIS`, cada uma implementando a interface `Imposto` com sua própria lógica de cálculo.
3. **Criar o Contexto:** Crie uma classe `CalculadoraDeImpostos` que possui um método `calcular(double valor, Imposto imposto)`. Este método simplesmente chama `imposto.calcula(valor)`.

Vantagem: A `CalculadoraDeImpostos` não conhece a lógica dos impostos. A lógica é delegada ao objeto da estratégia. Adicionar um novo imposto significa apenas criar uma nova classe.



Exercício Prático 02: Fretes com Strategy

Objetivo: Implementar o padrão Strategy.

Enunciado:

1. Crie uma interface `TipoDeFrete` com um método `double calcular(double distancia)`.
2. Crie três implementações: `FreteNormal` ($\text{distancia} * \text{R\$ } 1,25$), `FreteExpresso` ($\text{distancia} * \text{R\$ } 2,50$) e `FreteRetirada` (custo fixo de $\text{R\$ } 0,00$).
3. Crie uma classe `CalculadoraDeFrete` com um método `public double calculaFrete(TipoDeFrete tipo, double distancia)`.
4. Na `App`, simule o cálculo de um frete para uma distância de 100km usando os três tipos de frete.

File

Edit

Selection

View

Go

Run

Terminal

Help

Java002

Java002

Launchpad

0

1

Java: Ready

EXPLORER

OPEN EDITORS

JAVA002

.vscode

bin

lib

src

exercicios

_01

_02

App.java

CalculadoraDeFrete.java

FreteExpresso.java

FreteNormal.java

FreteRetirada.java

TipoDeFrete.java

oo

App.java

README.md

OUTLINE

TIMELINE

JAVA PROJECTS

TipoDeFrete.java

src > exercicios > _02 > TipoDeFrete.java > TipoDeFrete

1 package exercicios._02;

2

3 /**

4 * Interface (Strategy) que define o contrato para os diferentes tipos de cálculo de frete

5 * Qualquer classe que implemente esta interface será uma estratégia de frete válida.

6 */

7 public interface TipoDeFrete {

8

9 /**

10 * Calcula o valor do frete com base em uma distância.

11 *

12 * @param distancia A distância em quilômetros para o cálculo.

13 * @return O valor final do frete.

14 */

15 double calcular(double distancia);

16 }

CalculadoraDeFrete.java

src > exercicios > _02 > CalculadoraDeFrete.java > CalculadoraDeFrete

1 package exercicios._02;

2

3 /**

4 * Classe de Contexto que utiliza uma estratégia de frete para realizar o cálculo.

5 * Ela não conhece os detalhes de como o cálculo é feito, apenas delega a responsabilidade

6 * para o objeto de estratégia que recebe.

7 */

8 public class CalculadoraDeFrete {

9

10 /**

11 * Executa o cálculo de frete utilizando a estratégia fornecida.

12 *

13 * @param tipo O objeto da estratégia de frete a ser usada.

14 * @param distancia A distância para o cálculo.

15 * @return O valor do frete calculado pela estratégia.

16 */

17 public double calculaFrete(TipoDeFrete tipo, double distancia) {

18 return tipo.calcular(distancia);

19 }

20 }

FreteNormal.java

src > exercicios > _02 > FreteNormal.java > FreteNormal

1 package exercicios._02;

2

3 /**

4 * Estratégia Concreta para o cálculo de um frete normal.

5 */

6 public class FreteNormal implements TipoDeFrete {

7

8 @Override

9 public double calcular(double distancia) {

10 // Lógica específica do frete normal: R\$ 1,25 por km.

11 return distancia * 1.25;

12 }

13 }

FreteExpresso.java

src > exercicios > _02 > FreteExpresso.java > FreteExpresso

1 package exercicios._02;

2

3 /**

4 * Estratégia Concreta para o cálculo de um frete expresso.

5 */

6 public class FreteExpresso implements TipoDeFrete {

7

8 @Override

9 public double calcular(double distancia) {

10 // Lógica específica do frete expresso: R\$ 2,50 por km.

11 return distancia * 2.50;

12 }

13 }

FreteRetirada.java

src > exercicios > _02 > FreteRetirada.java > FreteRetirada

1 package exercicios._02;

2

3 /**

4 * Estratégia Concreta para retirada no local, com custo zero.

5 */

6 public class FreteRetirada implements TipoDeFrete {

7

8 @Override

9 public double calcular(double distancia) {

10 // Lógica específica para retirada: custo sempre zero, a distância

11 return 0.0;

12 }

13 }

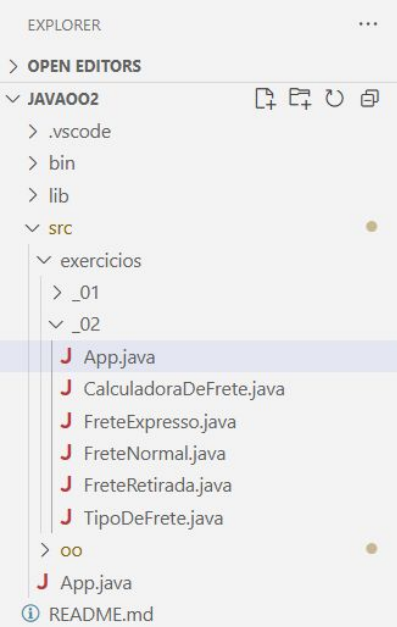
Ln 8, Col 34

Spaces: 4

UTF-8

CRLF

Java



```
J App.java x
src > exercicios > _02 > J App.java > {} exercicios._02
1 package exercicios._02;
2
3 public class App {
4
5     Run | Debug
6     public static void main(String[] args) {
7         // O cliente (nosso método main) controla qual estratégia usar.
8         CalculadoraDeFrete calculadora = new CalculadoraDeFrete();
9         double distancia = 100.0; // Distância de 100km para o teste
10
11        // 1. Instanciando as estratégias concretas
12        TipoDeFrete freteNormal = new FreteNormal();
13        TipoDeFrete freteExpresso = new FreteExpresso();
14        TipoDeFrete freteRetirada = new FreteRetirada();
15
16        // 2. Usando a calculadora com a estratégia de Frete Normal
17        double valorNormal = calculadora.calculaFrete(freteNormal, distancia);
18        System.out.printf(format:"O valor do frete Normal para %.1fkm é: R$ %.2f%n", distancia, valorNormal);
19
20        // 3. Usando a mesma calculadora com a estratégia de Frete Expresso
21        double valorExpresso = calculadora.calculaFrete(freteExpresso, distancia);
22        System.out.printf(format:"O valor do frete Expresso para %.1fkm é: R$ %.2f%n", distancia, valorExpresso);
23
24        // 4. Usando a mesma calculadora com a estratégia de Retirada
25        double valorRetirada = calculadora.calculaFrete(freteRetirada, distancia);
26        System.out.printf(format:"O valor do frete por Retirada para %.1fkm é: R$ %.2f%n", distancia, valorRetirada);
27    }
}
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\rlarg\OneDrive\Documentos\Java\Java002> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsI
lrg\OneDrive\Documentos\Java\Java002\bin' 'exercicios._02.App'
● O valor do frete Normal para 100,0km é: R$ 125,00
O valor do frete Expresso para 100,0km é: R$ 250,00
O valor do frete por Retirada para 100,0km é: R$ 0,00
○ PS C:\Users\rlarg\OneDrive\Documentos\Java\Java002>
```

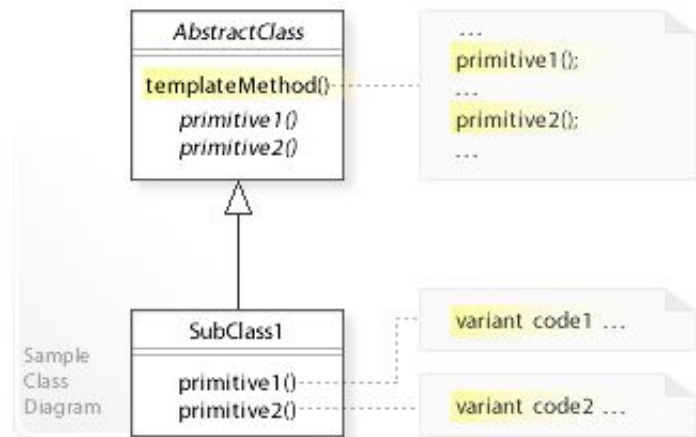
Aplicando Classes Abstratas: "Template Method"

Problema: Temos vários relatórios que seguem os mesmos passos (gerar cabeçalho, gerar corpo, gerar rodapé), mas o conteúdo do **corpo** de cada relatório é diferente.

Solução com Template Method:

1. **Definir o Template:** Crie uma classe abstrata `Relatorio` com um Método `final public void gerar()`.
2. **Estruturar o Algoritmo:** O método `gerar()` chama, em ordem, os métodos: `gerarCabeçalho()`, `gerarCorpo()` e `gerarRodape()`.
3. **Partes Fixas e Variáveis:** `gerarCabeçalho()` e `gerarRodape()` são métodos concretos (iguais para todos). `gerarCorpo()` é um método **abstrato**, forçando as subclasses a definirem o conteúdo.

Vantagem: A estrutura do algoritmo é fixa na superclasse, mas as subclasses podem "preencher as lacunas" de forma especializada, sem quebrar a ordem dos passos.



Exercício Prático 03: Processador de Documentos

Objetivo: Implementar o padrão Template Method.

Enunciado:

1. Crie uma classe abstrata `ProcessadorDeDocumento` com um método `final public void processar()`.
2. O método `processar()` deve chamar, na ordem: `abrir()`, `formatar()` e `fechar()`.
3. Os métodos `abrir()` e `fechar()` são concretos e imprimem "Abrindo documento..." e "Fechando documento..".
4. O método `formatar()` é **abstrato**.
5. Crie duas subclasses: `ProcessadorDOCX` e `ProcessadorPDF`.
6. Implemente o método `formatar()` em cada uma para imprimir "Formatando conteúdo para DOCX..." e "Formatando conteúdo para PDF..".
7. Na `App`, instancie e execute o processamento para os dez documentos variando os tipos.

The image shows a Java IDE with four editor tabs: `ProcessadorArquivo.java`, `App.java`, `ProcessadorXml.java`, and `ProcessadorTxt.java`. The `ProcessadorArquivo.java` file defines an abstract class with a template method and three concrete subclasses. The `App.java` file demonstrates the usage of these classes. The `ProcessadorXml.java` and `ProcessadorTxt.java` files are concrete subclasses that implement the `lerConteudo()` method.

```
package exercicios._03;

/**
 * Classe Abstrata (Template) que define o esqueleto de um algoritmo
 * para processar um arquivo. O método "processar()" é o "Template Method".
 */
public abstract class ProcessadorArquivo {

    /**
     * Este é o Template Method. Ele é 'final' para garantir que as subclasses
     * não possam alterar a sequência de passos do algoritmo.
     */
    public final void processar() {
        // Passo 1: Fixo, implementado na classe base.
        abrirArquivo();
        // Passo 2: Variável, deve ser implementado pelas subclasses.
        lerConteudo();
        // Passo 3: Fixo, implementado na classe base.
        fecharArquivo();
    }

    private void abrirArquivo() {
        System.out.println(x:"Abrindo arquivo...");
    }

    private void fecharArquivo() {
        System.out.println(x:"Fechando arquivo...");
    }

    /**
     * Este método é a parte do algoritmo que varia e deve ser
     * implementado por cada subclasse concreta.
     */
    protected abstract void lerConteudo();
}

package exercicios._03;

/**
 * Subclasse Concreta que implementa a parte variável do algoritmo
 * para ler arquivos do tipo XML.
 */
public class ProcessadorXml extends ProcessadorArquivo {

    @Override
    protected void lerConteudo() {
        System.out.println(x:">> Lendo e processando o conteúdo do arquivo XML.");
    }
}

package exercicios._03;

/**
 * Subclasse Concreta que implementa a parte variável do algoritmo
 * para ler arquivos do tipo TXT.
 */
public class ProcessadorTxt extends ProcessadorArquivo {

    @Override
    protected void lerConteudo() {
        System.out.println(x:">> Lendo e processando o conteúdo do arquivo TXT.");
    }
}

package exercicios._03;

public class App {

    public static void main(String[] args) {
        System.out.println(x:"--- Iniciando processamento de um arquivo TXT ---");
        // Instancia o processador de TXT
        ProcessadorArquivo processadorTxt = new ProcessadorTxt();
        // Chama o Template Method. Ele executará os 3 passos na ordem correta.
        processadorTxt.processar();

        System.out.println(x:"\n-----\n");

        System.out.println(x:"--- Iniciando processamento de um arquivo XML ---");
        // Instancia o processador de XML
        ProcessadorArquivo processadorXml = new ProcessadorXml();
        // Chama o mesmo Template Method, mas o resultado do passo 2 será diferente.
        processadorXml.processar();
    }
}
```

The terminal output shows the execution of the `App` class, demonstrating the template method pattern. It shows the execution of the `processar()` method for both `ProcessadorTxt` and `ProcessadorXml` classes, which inherit from `ProcessadorArquivo`.

```
PS C:\Users\rnlang\OneDrive\Documentos\Java\Java002> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rnlang\OneDrive\Documentos\Java\Java002\bin' 'exercicios._03.App'
--- Iniciando processamento de um arquivo TXT ---
Abrindo arquivo...
>> Lendo e processando o conteúdo do arquivo TXT.
Fechando arquivo...
>> Lendo e processando o conteúdo do arquivo TXT.
>> Lendo e processando o conteúdo do arquivo TXT.
>> Lendo e processando o conteúdo do arquivo TXT.
>> Lendo e processando o conteúdo do arquivo TXT.
Fechando arquivo...
```


PARA FIXAR: Evoluindo o Caixa Eletrônico

Objetivo: Consolidar todos os conceitos da aula.

Descrição:

1. Transforme a classe `Conta` da aula anterior em uma **classe abstrata**. Adicione um método abstrato `imprimirExtrato()`.
2. Crie uma **interface** `Tributavel` com um método `double getValorImposto()`.
3. Faça `ContaCorrente` herdar de `Conta` e implementar `Tributavel`. O imposto é 0.5% do valor de cada saque.
4. Faça `ContaPoupanca` apenas herdar de `Conta`.
5. Implemente `imprimirExtrato()` nas duas classes para exibir o saldo e informações específicas (ex: "Conta Corrente com taxa de saque").
6. Crie uma classe `SeguroDeVida` que **não herda de `Conta`**, mas implementa `Tributavel` com um valor fixo de imposto (ex: R\$ 42,00).
7. Na `App`, crie um `ArrayList` de `Tributavel`. Adicione instâncias de `ContaCorrente` e `SeguroDeVida`. Some os tributos de todos os itens da lista e exiba o total.



Aula 07 - 31/03/2026

Tratamento de Exceções

Sumário

- Introdução ao Tratamento de Exceções
 - O que são exceções e por que tratá-las? Diferenças entre exceções, erros e falhas comuns em Java.
- Conceitos Fundamentais
 - Tipos de exceções: Checked vs. Unchecked; Blocos try-catch-finally: Sintaxe e uso; Lançando exceções com throw e exceções personalizadas.
- Exercício Prático 01: Tratamento Básico de Entrada de Dados
- Design Patterns em Ação
 - Strategy para estratégias de recuperação de erros; Template Method para fluxos de tratamento padronizados.
- Exercício Prático 02: Aplicando Strategy em um Processador de Arquivos
- Boas Práticas e Avançado
 - Logs, relançamento e tratamento específico; Exceções em coleções e multithreading (vislumbre).
- Exercício Prático 03: Evoluindo o Caixa Eletrônico com Exceções
- Para Fixar: Integração com Conceitos Anteriores

Introdução ao Tratamento de Exceções

- **O que são exceções?**

- Eventos que interrompem o fluxo normal de um programa, como divisão por zero ou arquivo não encontrado. Em **Java**, elas são objetos da hierarquia **Throwable** (superclasse de **Exception** e **Error**).
- **Por que tratar?** Para tornar o código robusto, evitar crashes e fornecer feedback amigável ao usuário. Sem tratamento, o programa para e mostra um stack trace!

- **Exceções vs. Erros**

- **Exceções** (**Exception**): Podem ser recuperadas (ex: **IOException** para falha em I/O).
- **Erros** (**Error**): Problemas graves do sistema, como **OutOfMemoryError** – geralmente não tratamos, só previmos.
- **Exemplo prático**: Imagine um app de banco. Uma exceção de saldo insuficiente é tratável; um erro de memória não é.

Exemplo de Código Básico (Sem Tratamento):

J ExemploSemTratamento.java X

src > J ExemploSemTratamento.java > ...

```
1 public class ExemploSemTratamento {  
    Run | Debug  
2     public static void main(String[] args) {  
3         int[] numeros = {1, 2, 3};  
4         System.out.println(numeros[5]); // ArrayIndexOutOfBoundsException!  
5     }  
6 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions\bin' 'ExemploSemTratamento'

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
at ExemploSemTratamento.main(ExemploSemTratamento.java:4)

PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> □

Conceitos Fundamentais

- **Tipos de Exceções**

- **Checked Exceptions:** Compilador força tratamento (ex: `IOException`). São para problemas previsíveis.
- **Unchecked Exceptions (`RuntimeException`):** Não obrigatórias (ex: `NullPointerException`, `ArithmeticException`). Para erros de programação.
- Exemplo: `SQLException` (checked) vs. `IllegalArgumentException` (unchecked).

- **Blocos Try-Catch-Finally**

- **Sintaxe:**

```
try { /*código arriscado*/ }  
catch (TipoExcecao e) { /*tratamento*/ }  
finally { /*sempre executa*/ }
```

- Múltiplos `catch`: Do mais específico ao genérico.
- `finally`: Para cleanup, como fechar recursos.

Exemplo: Tratando IOException ao Ler Arquivo

J LeitorArquivo.java ×

src > J LeitorArquivo.java > LeitorArquivo

```
1  import java.io.*;
2
3  public class LeitorArquivo {
    Run | Debug
4      public static void main(String[] args) {
5          try (BufferedReader br = new BufferedReader(new FileReader(fileName:"arquivo.txt"))) {
6              String linha = br.readLine();
7              System.out.println(linha);
8          } catch (FileNotFoundException e) {
9              System.out.println("Arquivo não encontrado: " + e.getMessage());
10         } catch (IOException e) {
11             System.out.println("Erro de I/O: " + e.getMessage());
12         } finally {
13             System.out.println(x:"Recursos limpos!");
14         }
15     }
16 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



```
PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions\bin' 'LeitorArquivo'
```

• Arquivo não encontrado: arquivo.txt (O sistema não pode encontrar o arquivo especificado)
Recursos limpos!

○ PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> █

Exemplo: Tratando IOException ao Ler Arquivo

Usamos try-with-resources pra auto-fechar o arquivo. Sem isso, é como deixar a torneira aberta e fingir que não vai alagar!

- **Lançando Exceções**

- Use `throw new ExcecaoTipo("mensagem")`.
- Exceções personalizadas: Estenda `Exception` ou `RuntimeException`.

Exemplo: Exceção Personalizada de Idade Inválida

```
src > J IdadeInvalidaException.java > IdadeInvalidaException
1 class IdadeInvalidaException extends Exception {
2     public IdadeInvalidaException(String mensagem) {
3         super(mensagem);
4     }
5 }
```

```
src > J ValidadorIdade.java > ValidadorIdade
1 public class ValidadorIdade {
2     public static void validarIdade(int idade) throws IdadeInvalidaException {
3         if (idade < 0 || idade > 150) {
4             throw new IdadeInvalidaException("Idade inválida: " + idade);
5         }
6         System.out.println(x:"Idade válida!");
7     }
8
9     Run | Debug
10    public static void main(String[] args) {
11        try {
12            validarIdade(-5);
13        } catch (IdadeInvalidaException e) {
14            System.out.println("Erro: " + e.getMessage());
15        }
16    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rlarg\vaExceptions\bin' 'ValidadorIdade'
Erro: Idade inválida: -5
○ PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions>
```

Exercício Prático 01: Validação de Dados

- **Objetivo:** Aplicar try-catch e exceções personalizadas em um validador de formulário.
- **Enunciado:**
 1. Crie classes de exceção: `EmailInvalidoException` e `IdadeInvalidaException`.
 2. Crie uma classe `ValidadorUsuario` com método `validar(String email, int idade)` que lança exceções se inválido (email sem '@', idade < 0).
 3. Na `App`, use a classe `Scanner` para coletar dados, e use try-catch para capturar e imprimir mensagens amigáveis.
 4. Teste com dados válidos e inválidos.
- **Dica para Fixar:** Integre com encapsulamento da Aula 04 – use getters/setters na classe `Usuario`.

Exercício Prático 01 (Solução)

J EmailInvalidoException.java ×

src > ex01 > J EmailInvalidoException.java > ...

```
1 package ex01;
2
3 public class EmailInvalidoException extends Exception {
4     public EmailInvalidoException(String mensagem) {
5         super(mensagem);
6     }
7 }
```

J IdadeInvalidaException.java ×

src > ex01 > J IdadeInvalidaException.java > IdadeInvalidaException

```
1 package ex01;
2
3 public class IdadeInvalidaException extends Exception {
4     public IdadeInvalidaException(String mensagem) {
5         super(mensagem);
6     }
7 }
```

J ValidadorUsuario.java ×

src > ex01 > J ValidadorUsuario.java > ValidadorUsuario

```
1 package ex01;
2
3 public class ValidadorUsuario {
4     public static void validar(String email, int idade) throws EmailInvalidoException, IdadeInvalidaException {
5         if (email == null || !email.contains("@")) {
6             throw new EmailInvalidoException("Email inválido: deve conter '@'. Valor fornecido: " + email);
7         }
8         if (idade < 0) {
9             throw new IdadeInvalidaException("Idade inválida: deve ser maior ou igual a 0. Valor fornecido: " + idade);
10        }
11        System.out.println("Usuário válido! Email: " + email + ", Idade: " + idade);
12    }
13 }
```

Exercício Prático 01 (Solução)

```
J Usuario.java x
src > ex01 > J Usuario.java > Usuario
1 package ex01;
2
3 public class Usuario {
4     private String email;
5     private int idade;
6
7     public Usuario(String email, int idade) {
8         this.email = email;
9         this.idade = idade;
10    }
11
12    public String getEmail() {
13        return email;
14    }
15
16    public void setEmail(String email) {
17        this.email = email;
18    }
19
20    public int getIdade() {
21        return idade;
22    }
23
24    public void setIdade(int idade) {
25        this.idade = idade;
26    }
27 }
```

```
J App.java x
src > ex01 > J App.java > ...
1 package ex01;
2
3 import java.util.Scanner;
4
5 public class App {
6     Run | Debug
7     public static void main(String[] args) {
8         Scanner scanner = new Scanner(System.in);
9
10        System.out.println(x:"--- Validador de Usuário ---");
11        System.out.println(x:"Digite o email e a idade. Vamos testar validação com exceções!");
12
13        while (true) {
14            System.out.print(s:"Email: ");
15            String email = scanner.nextLine();
16
17            System.out.print(s:"Idade: ");
18            int idade = -1; // Valor padrão pra caso dê erro na leitura
19            try {
20                idade = Integer.parseInt(scanner.nextLine());
21            } catch (NumberFormatException e) {
22                System.out.println(x:"Erro: Idade deve ser um número inteiro. Tente novamente.");
23                continue; // Volta pro loop
24            }
25
26            // Cria o usuário (usando encapsulamento)
27            Usuario usuario = new Usuario(email, idade);
28
29            // Tenta validar - aqui entra o try-catch pro tratamento
30            try {
31                ValidadorUsuario.validar(usuario.getEmail(), usuario.getIdade());
32                System.out.println(x:"Sucesso! Usuário criado.");
33            } catch (EmailInvalidoException e) {
34                System.out.println("Ops! " + e.getMessage() + " Tente corrigir o email.");
35            } catch (IdadeInvalidaException e) {
36                System.out.println("Ops! " + e.getMessage() + " Tente corrigir a idade.");
37            }
38
39            System.out.print(s:"Quer testar outro? (s/n): ");
40            String resposta = scanner.nextLine();
41            if (!resposta.equalsIgnoreCase(anotherString:"s")) {
42                break;
43            }
44        }
45
46        System.out.println(x:"Fim do teste! Lembre-se: exceções tornam o código mais robusto.");
47        scanner.close();
48    }
}
```

```
PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> & 'C:\Program Files\OpenJDK\jdk-22.0.2'
os\Java\JavaExceptions\bin' 'ex01.App'
```

● --- Validador de Usuário ---

Digite o email e a idade. Vamos testar validação com exceções!

Email: ricardo@teste.com

Idade: 42

Usuário válido! Email: ricardo@teste.com, Idade: 42

Sucesso! Usuário criado.

Quer testar outro? (s/n): s

Email: ricardo-teste.com

Idade: 46

Ops! Email inválido: deve conter '@'. Valor fornecido: ricardo-teste.com Tente corrigir o email.

Quer testar outro? (s/n): s

Email: ricardo@teste.com

Idade: -5

Ops! Idade inválida: deve ser maior ou igual a 0. Valor fornecido: -5 Tente corrigir a idade.

Quer testar outro? (s/n): n

Fim do teste! Lembre-se: exceções tornam o código mais robusto.

○ PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaExceptions> █

Exercício Prático 01: Solução

Design Patterns em Ação

- **Strategy (com Interfaces para Tratamento Dinâmico)**
 - **Problema:** Diferentes exceções precisam de respostas variadas (ex: retry para rede, log para validação). Evite if-else gigante!
 - **Solução:** Interface `EstrategiaTratamento` com método `tratar(Exception e)`. Classes como `RetryStrategy` (tenta de novo) e `LogStrategy` (registra e continua).
 - **Exemplo Contextual:** Em um app de API, escolha a estratégia baseada na exceção.

J EstrategiaTratamento.java X

src > J EstrategiaTratamento.java > EstrategiaTratamento

```
1 interface EstrategiaTratamento {
2     void tratar(Exception e);
3 }
```

J RetryStrategy.java X

src > J RetryStrategy.java > RetryStrategy > tratar(Exception)

```
1 class RetryStrategy implements EstrategiaTratamento {
2     @Override
3     public void tratar(Exception e) {
4         System.out.println("Tentando novamente... " + e.getMessage());
5         // Lógica de retry aqui
6     }
7 }
```

J LogStrategy.java X

src > J LogStrategy.java > LogStrategy

```
1 class LogStrategy implements EstrategiaTratamento {
2     @Override
3     public void tratar(Exception e) {
4         System.out.println("Logando erro: " + e.getMessage());
5         // Escreve em log
6     }
7 }
```

J GerenciadorExcecoes.java X

src > J GerenciadorExcecoes.java > GerenciadorExcecoes

```
1 import java.io.IOException;
2
3 public class GerenciadorExcecoes {
4     private EstrategiaTratamento estrategia;
5
6     public void setEstrategia(EstrategiaTratamento estrategia) {
7         this.estrategia = estrategia;
8     }
9
10    public void processar() {
11        try {
12            // Código que pode falhar
13            throw new IOException(message:"Falha de rede");
14        } catch (Exception e) {
15            estrategia.tratar(e);
16        }
17    }
18 }
```

Exemplo: Strategy para Exceções de Rede

Design Patterns em Ação

- **Template Method** (com Classes Abstratas para Fluxo de Tratamento)
 - **Problema:** Todo tratamento segue passos (capturar, logar, recuperar, notificar), mas detalhes variam por tipo de exceção.
 - **Solução:** Classe abstrata `TratadorExcecao` com método final `tratar()` que chama `capturar()`, `recuperar()` (abstrato) e `notificar()`.
 - **Exemplo Contextual:** Para exceções em I/O vs. validação.

Exemplo: Template Method com Classes Abstratas para Fluxo de Tratamento

J TratadorExcecao.java X

src > J TratadorExcecao.java > TratadorExcecao

```
1 abstract class TratadorExcecao {
2     public final void tratar(Exception e) {
3         capturar(e);
4         recuperar(e);
5         notificar(e);
6     }
7
8     private void capturar(Exception e) {
9         System.out.println("Capturando: " + e.getMessage());
10    }
11
12    protected abstract void recuperar(Exception e);
13
14    private void notificar(Exception e) {
15        System.out.println("Notificando usuário sobre: " + e.getClass().getSimpleName());
16    }
17 }
```

J TratadorIO.java X

src > J TratadorIO.java > TratadorIO > recuperar(Exception)

```
1 class TratadorIO extends TratadorExcecao {
2     @Override
3     protected void recuperar(Exception e) {
4         System.out.println(x:"Recuperando arquivo backup...");
5     }
6 }
```

J TratadorValidacao.java X

src > J TratadorValidacao.java > TratadorValidacao

```
1 class TratadorValidacao extends TratadorExcecao {
2     @Override
3     protected void recuperar(Exception e) {
4         System.out.println(x:"Solicitando dados corretos...");
5     }
6 }
```

Exercício Prático 02: Aplicando Strategy em um Processador de Arquivos

- **Objetivo:** Refatorar um leitor de arquivos usando Strategy para tratar exceções de forma intercambiável.
- **Enunciado:**
 1. Crie interface `EstrategiaRecuperacao` com `recuperar(IOException e)`.
 2. Implementações: `RetryRecuperacao` (tenta 3x) e `AbortRecuperacao` (para e loga).
 3. Classe `ProcessadorArquivo` que usa a estratégia no try-catch.
 4. Na `App`, teste com arquivo inexistente, alternando estratégias.
- Pense no Strategy como um "plano B flexível"
 1. se uma falha, mude de tática sem drama!

```
J EstrategiaRecuperacao.java ×
src > ex02 > J EstrategiaRecuperacao.java > ...
1 package ex02;
2 import java.io.IOException;
3
4 public interface EstrategiaRecuperacao {
5     void recuperar(IOException e);
6 }
```

```
J RetryRecuperacao.java ×
src > ex02 > J RetryRecuperacao.java > RetryRecuperacao > recuperar(IOException)
1 package ex02;
2 import java.io.IOException;
3
4 public class RetryRecuperacao implements EstrategiaRecuperacao {
5     @Override
6     public void recuperar(IOException e) {
7         System.out.println(x:"Tentando novamente... (simulando 3 tentativas)");
8         for (int i = 1; i <= 3; i++) {
9             System.out.println("Tentativa " + i + ": Falhou com " + e.getMessage());
10            // Aqui você poderia tentar ler o arquivo novamente, mas pra simplicidade, só simula
11        }
12        System.out.println("Após 3 tentativas, abortando. Logando erro: " + e.getMessage());
13    }
14 }
```

```
J AbortRecuperacao.java ×
src > ex02 > J AbortRecuperacao.java > AbortRecuperacao
1 package ex02;
2 import java.io.IOException;
3
4 public class AbortRecuperacao implements EstrategiaRecuperacao {
5     @Override
6     public void recuperar(IOException e) {
7         System.out.println("Abortando operação. Logando erro: " + e.getMessage());
8         // Poderia escrever em um arquivo de log real aqui
9     }
10 }
```

Exercício Prático 02 (Solução)

```
1 package ex02;
2
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5 import java.io.IOException;
6
7 public class ProcessadorArquivo {
8     private EstrategiaRecuperacao estrategia;
9
10    public ProcessadorArquivo(EstrategiaRecuperacao estrategia) {
11        this.estrategia = estrategia;
12    }
13
14    public void setEstrategia(EstrategiaRecuperacao estrategia) {
15        this.estrategia = estrategia;
16    }
17
18    public void processarArquivo(String caminho) {
19        try (BufferedReader br = new BufferedReader(new FileReader(caminho))) {
20            String linha;
21            while ((linha = br.readLine()) != null) {
22                System.out.println("Linha lida: " + linha);
23            }
24            System.out.println("Arquivo processado com sucesso!");
25        } catch (IOException e) {
26            System.out.println("Erro ao processar arquivo: " + e.getMessage());
27            estrategia.recuperar(e); // Delega pra estratégia
28        }
29    }
30 }
```

Exercício Prático 02 (Solução)

```

AppJava x
src > ex02 > J AppJava > App > main(String[])
1 package ex02;
2
3 import java.util.Scanner;
4
5 public class App {
6     Run | Debug
7     public static void main(String[] args) {
8         Scanner scanner = new Scanner(System.in);
9         EstrategiaRecuperacao estrategiaAtual = new RetryRecuperacao(); // Começa com retry
10        ProcessadorArquivo processador = new ProcessadorArquivo(estrategiaAtual);
11
12        System.out.println(x:"--- Processador de Arquivos com Strategy ---");
13        System.out.println(x:"Vamos testar com um arquivo inexistente: 'arquivo_inexistente.txt'");
14
15        while (true) {
16            System.out.println("\nEstratégia atual: " + estrategiaAtual.getClass().getSimpleName());
17            System.out.println(x:"1. Processar arquivo (usando estratégia atual)");
18            System.out.println(x:"2. Trocar para RetryRecuperacao");
19            System.out.println(x:"3. Trocar para AbortRecuperacao");
20            System.out.println(x:"4. Sair");
21            System.out.print(s:"Escolha: ");
22
23            int opcao = scanner.nextInt();
24            scanner.nextLine(); // Consumir quebra de linha
25
26            switch (opcao) {
27                case 1:
28                    processador.processarArquivo(caminho:"arquivo_inexistente.txt");
29                    break;
30                case 2:
31                    estrategiaAtual = new RetryRecuperacao();
32                    processador.setEstrategia(estrategiaAtual);
33                    System.out.println(x:"Estratégia trocada para Retry!");
34                    break;
35                case 3:
36                    estrategiaAtual = new AbortRecuperacao();
37                    processador.setEstrategia(estrategiaAtual);
38                    System.out.println(x:"Estratégia trocada para Abort!");
39                    break;
40                case 4:
41                    System.out.println(x:"Saindo...");
42                    scanner.close();
43                    return;
44                default:
45                    System.out.println(x:"Opção inválida!");
46            }
47        }
48    }
}

```

```

CHAT
Run: App + v [ ] [ ] ...
PS C:\Users\r\lang\OneDrive\Documents\Java\JavaExceptions> & 'C:\Program Files\OpenDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\r\lang\OneDrive\Documents\Java\JavaExceptions\bin' 'ex02.App'
--- Processador de Arquivos com Strategy ---
Vamos testar com um arquivo inexistente: 'arquivo_inexistente.txt'

Estratégia atual: RetryRecuperacao
1. Processar arquivo (usando estratégia atual)
2. Trocar para RetryRecuperacao
3. Trocar para AbortRecuperacao
4. Sair
Escolha: 1
Erro ao processar arquivo: arquivo_inexistente.txt (O sistema não pode encontrar o arquivo especificado)
Tentando novamente... (simulando 3 tentativas)
Tentativa 1: Falhou com arquivo_inexistente.txt (O sistema não pode encontrar o arquivo especificado)
Tentativa 2: Falhou com arquivo_inexistente.txt (O sistema não pode encontrar o arquivo especificado)
Tentativa 3: Falhou com arquivo_inexistente.txt (O sistema não pode encontrar o arquivo especificado)
Após 3 tentativas, abortando. Logando erro: arquivo_inexistente.txt (O sistema não pode encontrar o arquivo especificado)

Estratégia atual: RetryRecuperacao
1. Processar arquivo (usando estratégia atual)
2. Trocar para RetryRecuperacao
3. Trocar para AbortRecuperacao
4. Sair
Escolha: 2
Estratégia trocada para Retry!

Estratégia atual: RetryRecuperacao
1. Processar arquivo (usando estratégia atual)
2. Trocar para RetryRecuperacao
3. Trocar para AbortRecuperacao
4. Sair
Escolha: 3
Estratégia trocada para Abort!

Estratégia atual: AbortRecuperacao
1. Processar arquivo (usando estratégia atual)
2. Trocar para RetryRecuperacao
3. Trocar para AbortRecuperacao
4. Sair
Escolha: 4
Saindo...
PS C:\Users\r\lang\OneDrive\Documents\Java\JavaExceptions>

```

Exercício Prático 02 (Solução)

Boas Práticas e Avançado

- **Logs e Relançamento:** Use `e.printStackTrace()` ou bibliotecas como SLF4J. Relance com `throw new MinhaExcecao("Detalhes", e);` pra encadear.
- **Tratamento Específico:** Catch o exato tipo primeiro. Evite `catch (Exception e)` – é preguiçoso e esconde bugs!
- **Avançado Rápido:** Em coleções, trate `IndexOutOfBoundsException`; em threads, `InterruptedException`.
- **Exemplo: Relançamento**

```
try {  
    // Código  
} catch (IOException e) {  
    throw new RuntimeException("Falha crítica de I/O", e);  
}
```


PARA FIXAR: Evoluindo o Caixa Eletrônico com Exceções

- **Objetivo:** Consolidar com conceitos anteriores (herança da Aula 06, encapsulamento da 04).
- **Descrição:**
 1. Na classe abstrata `Conta` (da Aula 06), adicione método `sacar(double valor)` que lança `SaldoInsuficienteException` (personalizada).
 2. Em `ContaCorrente` e `ContaPoupanca`, implemente com try-catch interno se precisar.
 3. Use Template Method para um `TratadorTransacao` que gerencie saques/depósitos.
 4. Na `App`, menu com tratamento: se exceção, use Strategy (ex: notificar ou retry).
 5. Integre `Tributavel`: Se exceção em imposto, logue e continue.
- **Teste:** Simule saques inválidos e veja o sistema gracioso!

